

A photograph of two SpaceX Falcon 9 first stages during a landing attempt. The stage on the left is in the process of landing, with its landing legs deployed and a large plume of fire and smoke at its base. The stage on the right is in the air, with its landing legs also deployed and a smaller plume of fire and smoke at its base. The background is a clear blue sky with some clouds.

SpaceX Falcon 9 first stage Landing Prediction

Shelby Dockery

Agenda

1. Executive Summary
2. Introduction
3. Data collection and data wrangling methodology
4. EDA and interactive visual analytics methodology
5. Predictive analysis methodology
6. EDA with visualization Results
7. EDA with SQL Results
8. Interactive map with Folium Results
9. Plotly Dash dashboard Results
10. Predictive analysis (classification) Results
11. Conclusion

Executive Summary

- This report presented the findings of a data analysis project about predicting the success of the first stage landing of the SpaceX Falcon 9 rocket. The reason behind this data analysis project was to assist a startup company in making informed bids against SpaceX for rocket launches.
- The project involved data collecting and data wrangling, performing exploratory data analysis, developing predictive models, and evaluating skills. The dataset contained historical data on the Falcon 9 launches, including various parameters such as launch site, weather conditions, and mission success.
- Based on the analysis, the predictive model using logistic regression that achieved an accuracy of 83% in predicting the success of the first stage landing. The model identified key factors that significantly influence the landing such as launch site, payload mass, orbit, flightnumber, and more.
- The findings of this analysis provide some knowledge for the startup company to make data-driven decisions when bidding against SpaceX. By using the predictive model, the company can assess the likelihood of a successful landing and adjust their bids accordingly.
- In conclusion, this data analysis project has demonstrated the potential of using predictive modeling to inform decision-making in the space industry. The findings and models developed in this project can be used as a valuable asset for the startup company to showcase their capabilities and compete effectively with SpaceX

Introduction

- Goal: To predict the landing of the first stage of the Falcon 9 in order to determine the cost of the launch
- Reason why: So, an alternate company like SpaceY can bid against SpaceX for a rocket launch
- Following code scripts that were utilized in this project can be found in the link below
 - [shelbydockery12/testrepo mod10 at Capstone-Project-Python-Scripts \(github.com\)](#)

Data collection and data wrangling methodology

- In order to collect the right data to complete with SpaceX, I used the following data collection and data wrangling methodology that are shown in these slides

Data collection and data wrangling Step 1

- First, I imported the following libraries in python

```
In [18]: # Requests allows us to make HTTP requests which we will use to get data from an API  
import requests  
# Pandas is a software library written for the Python programming language for data manipulation and analysis.  
import pandas as pd  
# NumPy is a library for the Python programming language, adding support for large, multi-dimensional arrays and matrices,  
import numpy as np  
# Datetime is a library that allows us to represent dates  
import datetime  
# Setting this option will print all columns of a dataframe
```

Data collection and data wrangling Step 2

- Next, I created a get request to retrieve the SpaceX data from the url below.

```
In [25]: static_json_url='https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/IBM-DS0321EN-SkillsNetwork/datasets/API_
```

```
In [26]: response = requests.get(static_json_url)
```

- After getting the data from the url, I converted the json file into a dataframe called data
 - As shown in the image below

```
In [28]: # Use json_normalize meethod to convert the json result into a dataframe  
data = pd.json_normalize(response.json())
```

Data collection and data wrangling Step 2

- Once the dataframe was created, I wanted to modify to where it only contained the data that was needed in order to predict the launches for SpaceX
- The following data that was important is in the variable data
 - As shown in the image below

```
In [30]: # Lets take a subset of our dataframe keeping only the features we want and the flight number, and date_utc.
data = data[['rocket', 'payloads', 'launchpad', 'cores', 'flight_number', 'date_utc']]

# We will remove rows with multiple cores because those are falcon rockets with 2 extra rocket boosters and rows that have n
data = data[data['cores'].map(len)==1]
data = data[data['payloads'].map(len)==1]

# Since payloads and cores are lists of size 1 we will also extract the single value in the list and replace the feature.
data['cores'] = data['cores'].map(lambda x : x[0])
data['payloads'] = data['payloads'].map(lambda x : x[0])

# We also want to convert the date_utc to a datetime datatype and then extracting the date leaving the time
data['date'] = pd.to_datetime(data['date_utc']).dt.date

# Using the date we will restrict the dates of the launches
data = data[data['date'] <= datetime.date(2020, 11, 13)]
```


Data collection and data wrangling Step 2 Continued

- After modifying the data to meet my needs, I realized majority of the data was IDs.
- Therefore, I had to use the API again along with the following functions to get the information from the launches
- The image below is the function I used to get the booster name from the rocket column

```
In [19]: # Takes the dataset and uses the rocket column to call the API and append the data to the list  
def getBoosterVersion(data):  
    for x in data['rocket']:  
        if x:  
            response = requests.get("https://api.spacexdata.com/v4/rockets/"+str(x)).json()  
            BoosterVersion.append(response['name'])
```

API Functions

- The image below is the function I used to get the booster name from the rocket column

```
In [19]: # Takes the dataset and uses the rocket column to call the API and append the data to the list
def getBoosterVersion(data):
    for x in data['rocket']:
        if x:
            response = requests.get("https://api.spacexdata.com/v4/rockets/"+str(x)).json()
            BoosterVersion.append(response['name'])
```

- The image below is the function I used to get the name of the launch site being used from the launchpad column

```
In [20]: # Takes the dataset and uses the launchpad column to call the API and append the data to the list
def getLaunchSite(data):
    for x in data['launchpad']:
        if x:
            response = requests.get("https://api.spacexdata.com/v4/launchpads/"+str(x)).json()
            Longitude.append(response['longitude'])
            Latitude.append(response['latitude'])
            LaunchSite.append(response['name'])
```

API Functions

- The image below is the function I used to get the mass of the payload along with the orbit that it is going to from the payload column

```
In [21]: # Takes the dataset and uses the payloads column to call the API and append the data to the lists
def getPayloadData(data):
    for load in data['payloads']:
        if load:
            response = requests.get("https://api.spacexdata.com/v4/payloads/"+load).json()
            PayloadMass.append(response['mass_kg'])
            Orbit.append(response['orbit'])
```

API Functions

- The image below is the function I used to get the outcome of the landing, type of landing, number of flights with that core, etc from the core column

In [22]:

```
# Takes the dataset and uses the cores column to call the API and append the data to the lists
def getCoreData(data):
    for core in data['cores']:
        if core['core'] != None:
            response = requests.get("https://api.spacexdata.com/v4/cores/"+core['core']).json()
            Block.append(response['block'])
            ReusedCount.append(response['reuse_count'])
            Serial.append(response['serial'])
        else:
            Block.append(None)
            ReusedCount.append(None)
            Serial.append(None)
    Outcome.append(str(core['landing_success'])+' '+str(core['landing_type']))
    Flights.append(core['flight'])
    GridFins.append(core['gridfins'])
    Reused.append(core['reused'])
    Legs.append(core['legs'])
    LandingPad.append(core['landpad'])
```

Data collection and data wrangling Step 2 Continued

- After using the functions to get the information from the IDs, I created a series of lists that the data will be stored in.

```
In [31]: #Global variables  
BoosterVersion = []  
PayloadMass = []  
Orbit = []  
LaunchSite = []  
Outcome = []  
Flights = []  
GridFins = []  
Reused = []  
Legs = []  
LandingPad = []  
Block = []  
ReusedCount = []  
Serial = []  
Longitude = []  
Latitude = []
```

Data collection and data wrangling Step 2 Continued

- Now, I call the function to store the data in the list
- As shown below, I call the `BoosterVersion` function and all of the rocket ID data will be stored in the `BoosterVersion` List

```
In [33]: # Call getBoosterVersion  
getBoosterVersion(data)
```

```
In [34]: BoosterVersion[0:5]
```

```
Out[34]: ['Falcon 1', 'Falcon 1', 'Falcon 1', 'Falcon 1', 'Falcon 9']
```

Data collection and data wrangling Step 2 Continued

- Now, I call the functions to store the data in the lists that shown in the previous slide
- As shown below, I call the BoosterVersion function and all of the rocket ID data will be stored in the BoosterVersion List

```
In [33]: # Call getBoosterVersion  
         getBoosterVersion(data)
```

```
In [34]: BoosterVersion[0:5]
```

```
Out[34]: ['Falcon 1', 'Falcon 1', 'Falcon 1', 'Falcon 1', 'Falcon 9']
```

Data collection and data wrangling Step 2 Continued

- As stated in the previous slide, I call the following functions to get the information from the IDs and store them in the lists

```
In [35]: # Call getLaunchSite  
getLaunchSite(data)
```

```
In [36]: # Call getPayloadData  
getPayloadData(data)
```

```
In [37]: # Call getCoreData  
getCoreData(data)
```


Data collection and data wrangling Step 2 Continued

- After getting the data and storing into several lists, I created a dictionary to combine the columns of data

```
In [38]: launch_dict = {'FlightNumber': list(data['flight_number']),
                        'Date': list(data['date']),
                        'BoosterVersion': BoosterVersion,
                        'PayloadMass': PayloadMass,
                        'Orbit': Orbit,
                        'LaunchSite': LaunchSite,
                        'Outcome': Outcome,
                        'Flights': Flights,
                        'GridFins': GridFins,
                        'Reused': Reused,
                        'Legs': Legs,
                        'LandingPad': LandingPad,
                        'Block': Block,
                        'ReusedCount': ReusedCount,
                        'Serial': Serial,
                        'Longitude': Longitude,
                        'Latitude': Latitude}
```

- Once the dictionary called launch_dict was created, I turned it into a dataframe

```
In [41]: # Create a data from launch_dict
df = pd.DataFrame(launch_dict)
print('launch_dict is now a df')
```

launch_dict is now a df

Data collection and data wrangling Step 3

- Once the data frame was created, I needed to remove any Falcon 1 launches from the data frame and keeping only Falcon 9 launches
- In order to keep only Falcon 9 launch data, I used a comparison operator to drop any Falcon 1 launch data and created a new data frame data_falcon9

➤ != means “not equal to “

In [45]:

```
# Hint data['BoosterVersion']!='Falcon 1'  
data_falcon9 = df[df['BoosterVersion']!='Falcon 1']  
print('new df only containing Falcon 9 data')
```

new df only containing Falcon 9 data

- I also had to reset the Flight Number column since I removed some values

In [47]:

```
data_falcon9.loc[:, 'FlightNumber'] = list(range(1, data_falcon9.shape[0]+1))  
data_falcon9
```

Data collection and data wrangling Step 4

- When taking a look at the Falcon 9 data frame, I noticed that both Payload and LandingPad had some missing values. However, the LandingPad none values were normal since it indicated landing pads were not use.

```
In [48]: data_falcon9.isnull().sum()
```

```
Out[48]: FlightNumber      0  
Date                      0  
BoosterVersion            0  
PayloadMass               5  
Orbit                     0  
LaunchSite                0  
Outcome                   0  
Flights                   0  
GridFins                  0  
Reused                    0  
Legs                      0  
LandingPad                26  
Block                     0  
ReusedCount               0  
Serial                    0  
Longitude                 0  
Latitude                  0  
dtype: int64
```

Data collection and data wrangling Step 4 Continued

- Therefore, I used the mean function to find the mean of the PayloadMass and used the replace the null values in the PayloadMass with the mean value

```
In [49]: # Calculate the mean value of PayloadMass column
print('getting the mean of PayloadMass to replace nun values')
payloadmass_mean = data_falcon9['PayloadMass'].mean()
# Replace the np.nan values with its mean value
data_falcon9['PayloadMass'].replace(np.nan,payloadmass_mean, inplace = True)
print('PayloadMass has values')
```

```
getting the mean of PayloadMass to replace nun values
PayloadMass has values
```

```
In [53]: nan_values = data_falcon9['PayloadMass'].isnull().sum()
print(f'PayloadMass has {nan_values} nun values')
```

```
PayloadMass has 0 nun values
```

Data collection and data wrangling Step 4 Continued

- After updating the PayloadMass column, I exported the data frame into an excel csv file

In [55]:

```
save_path = 'dataset_part_1.csv'
```

```
data_falcon9.to_csv(save_path, index=False)
```

```
data_falcon9.to_csv('dataset_part_1.csv', index=False)
```

Data collection and data wrangling Step 5

- Once that file was converted into a csv file, I wanted to see if any independent variables such as flight number, payload mass, etc would affect the success rate of the rocket, however, most of the results for this step are shown and explained in the EDA with visualization Results slides
- Therefore, if you are interested in the outcomes, please visit that section before proceeding.
- Now, I will be skipping heading and explaining what I did next after noticing several factors that affect the success rate of the rocket.

Data collection and data wrangling Step 5 Continued

- After noticing the following features (shown in the image below) that affect the success rate of the rocket, I decided to take those columns from the csv file and convert them into a separate data frame called features

```
[3]: from js import fetch
import io

URL = "https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/IBM-DS0321EN-SkillsNetwork/datasets/dataset_part_2.csv"
resp = await fetch(URL)
dataset_part_2_csv = io.BytesIO((await resp.arrayBuffer()).to_py())
df=pd.read_csv(dataset_part_2_csv)
df.head(5)
```

```
[6]: features = df[['FlightNumber', 'PayloadMass', 'Orbit', 'LaunchSite', 'Flights', 'GridFins', 'Reused', 'Legs', 'LandingPad', 'Block', 'ReusedCount', 'Serial']]
features.head()
```

	FlightNumber	PayloadMass	Orbit	LaunchSite	Flights	GridFins	Reused	Legs	LandingPad	Block	ReusedCount	Serial
0	1	6104.959412	LEO	CCAFS SLC 40	1	False	False	False	NaN	1.0	0	B0003
1	2	525.000000	LEO	CCAFS SLC 40	1	False	False	False	NaN	1.0	0	B0005
2	3	677.000000	ISS	CCAFS SLC 40	1	False	False	False	NaN	1.0	0	B0007
3	4	500.000000	PO	VAFB SLC 4E	1	False	False	False	NaN	1.0	0	B1003
4	5	3170.000000	GTO	CCAFS SLC 40	1	False	False	False	NaN	1.0	0	B1004

Data collection and data wrangling Step 5 Continued

- After creating a new data frame called features, I used the function `get_dummies` in order to convert the categorical columns into indicator variables and created a new data frame called `features_one_hot`. As shown in the image below

```
[9]: # HINT: Use get_dummies() function on the categorical columns
cols = ['Orbit', 'LaunchSite', 'LandingPad', 'Serial']
features_one_hot = pd.get_dummies(features, columns=cols)
features_one_hot.head()
```

	FlightNumber	PayloadMass	Flights	GridFins	Reused	Legs	Block	ReusedCount	Orbit_ES-L1	Orbit_GEO	...	Serial_B1048	Serial_B1049	Serial_B1050	S
0	1	6104.959412	1	False	False	False	1.0	0	0	0	...	0	0	0	
1	2	525.000000	1	False	False	False	1.0	0	0	0	...	0	0	0	
2	3	677.000000	1	False	False	False	1.0	0	0	0	...	0	0	0	
3	4	500.000000	1	False	False	False	1.0	0	0	0	...	0	0	0	
4	5	3170.000000	1	False	False	False	1.0	0	0	0	...	0	0	0	

5 rows × 80 columns

Data collection and data wrangling Step 6

- Now that features_one_hot is all numerical, I simply converted the data frame into a float data frame for more precision and converted into a csv file.
- Also, when it comes to the prediction analysis methodology slides this data frame will be our independent variables while our other data frame called data_falcon9 will be our dependent variable since it has the Class column.

```
[10]: # HINT: use astype function
features_one_hot.astype('float64')
print(type(features_one_hot))
features_one_hot

<class 'pandas.core.frame.DataFrame'>
```

	FlightNumber	PayloadMass	Flights	GridFins	Reused	Legs	Block	ReusedCount	Orbit_ES-L1	Orbit_GEO	...	...
0	1	6104.959412	1	False	False	False	1.0	0	0	0	...	...
1	2	525.000000	1	False	False	False	1.0	0	0	0	...	...
2	3	677.000000	1	False	False	False	1.0	0	0	0	...	...
3	4	500.000000	1	False	False	False	1.0	0	0	0	...	...
4	5	3170.000000	1	False	False	False	1.0	0	0	0	...	...
...	...	...	...	...	...	...	...	...	...	...	...	...
85	86	15400.000000	2	True	True	True	5.0	2	0	0	...	...
86	87	15400.000000	3	True	True	True	5.0	2	0	0	...	...
87	88	15400.000000	6	True	True	True	5.0	5	0	0	...	...
88	89	15400.000000	3	True	True	True	5.0	2	0	0	...	...
89	90	3681.000000	1	True	False	True	5.0	0	0	0	...	...

90 rows x 80 columns

```
[ ]: features_one_hot.to_csv('dataset_part_3.csv', index=False)
```

EDA and interactive visual analytics methodology

- After creating a csv file called dataset part 3 that contained a list of the factors that could affect the success landing rate of the rocket, I realized that there must be a reason why SpaceX chose these launch sites as well.
- Therefore, I wanted to see what patterns I could find when looking at all of the launch sites. I utilized the following python packages in this data analysis.
 - Folium
 - Plotly
- The following slides are steps on how I performed my interactive visual analytics.
- If you are interested in the results of this data analysis, please skip ahead to Interactive map with Folium Results and Plotly Dash dashboard results.

EDA and interactive visual analytics methodology

Folium : Step 1

- Since I wanted to get a visualization on where these launch sites were located, I decided to use folium first.
- Like the previous methodology, I first imported the following libraries into my python code

In [2]:

```
import piplite
await piplite.install(['folium'])
await piplite.install(['pandas'])
```

In [3]:

```
import folium
import pandas as pd
```

In [4]:

```
# Import folium MarkerCluster plugin
from folium.plugins import MarkerCluster
# Import folium MousePosition plugin
from folium.plugins import MousePosition
# Import folium DivIcon plugin
from folium.features import DivIcon
```

EDA and interactive visual analytics methodology

Folium : Step 2

- Next, I read the csv file into a data frame and grouped them by launch in order to get the coordinates of each launch sight

```
In [6]: # Download and read the `spacex_launch_geo.csv`
from js import fetch
import io

URL = 'https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/IBM-DS0321EN-SkillsNetwork/datasets/spacex_launch_geo/spacex_launch_geo.csv'
resp = await fetch(URL)
spacex_csv_file = io.BytesIO((await resp.arrayBuffer()).to_py())
spacex_df = pd.read_csv(spacex_csv_file)
```

Now, you can take a look at what are the coordinates for each site.

```
In [7]: # Select relevant sub-columns: `Launch Site`, `Lat(Latitude)`, `Long(Longitude)`, `class`
spacex_df = spacex_df[['Launch Site', 'Lat', 'Long', 'class']]
launch_sites_df = spacex_df.groupby(['Launch Site'], as_index=False).first()
launch_sites_df = launch_sites_df[['Launch Site', 'Lat', 'Long']]
launch_sites_df
```

```
Out[7]:
```

	Launch Site	Lat	Long
0	CCAFS LC-40	28.562302	-80.577356
1	CCAFS SLC-40	28.563197	-80.576820
2	KSC LC-39A	28.573255	-80.646895
3	VAFB SLC-4E	34.632834	-120.610745

EDA and interactive visual analytics methodology

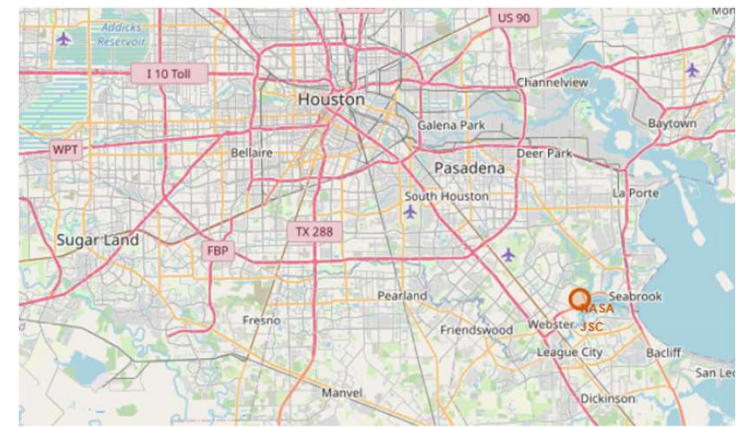
Folium : Step 3

- Once the launch site data frame was created, I created a folium map using the location of the NASA Johnson Space Center at Houston, Texas as the as the folium map object

```
In [8]: # Start Location is NASA Johnson Space Center
nasa_coordinate = [29.559684888503615, -95.0830971930759]
site_map = folium.Map(location=nasa_coordinate, zoom_start=10)
```

We could use `folium.Circle` to add a highlighted circle area with a text label on a specific coordinate. For example,

```
In [9]: # Create a blue circle at NASA Johnson Space Center's coordinate with a popup Label showing its name
circle = folium.Circle(nasa_coordinate, radius=1000, color='#d35400', fill=True).add_child(folium.Popup('NASA Johnson Space
# Create a blue circle at NASA Johnson Space Center's coordinate with a icon showing its name
marker = folium.map.Marker(
    nasa_coordinate,
    # Create an icon as a text label
    icon=DivIcon(
        icon_size=(20,20),
        icon_anchor=(0,0),
        html='<div style="font-size: 12; color:#d35400;"><b>%s</b></div>' % 'NASA JSC',
    )
)
site_map.add_child(circle)
site_map.add_child(marker)
```



EDA and interactive visual analytics methodology

Folium : Step 4

- After creating my folium map object, I next created a series of markers of each of the launch sites onto the map
- In order to make it easier on me, I also created a list that contained each of the launch sites coordinates instead of manually typing them in.

```
launchsites_lists = []
for index, row in launch_sites_df.iterrows():
    row_list = row.tolist()
    mod_list = row_list[1:]
    launchsites_lists.append(mod_list)
print(launchsites_lists)

[[28.56230197, -80.57735648], [28.56319718, -80.57682003], [28.57325457, -80.64689529], [34.63283416, -120.6107455]]
```

```
[ ]: # Initial the map
site_map = folium.Map(location=nasa_coordinate, zoom_start=5)
# For each launch site, add a Circle object based on its coordinate (lat, long) values. In addition, add launch site name as a popup label
# Initial the map

# For each launch site, add a Circle object based on its coordinate (lat, long) values. In addition, add launch site name as a popup label
circle = folium.Circle(launchsites_lists[0], radius=1000, color='red', fill = True).add_child(folium.Popup('CCAFS SLC-40'))
marker = folium.Marker(launchsites_lists[0], icon=DivIcon(icon_size=(20,20),icon_anchor=(0,0), html='<div style="font-size: 12; color:red"><b>X</b></div>'),
site_map.add_child(circle)
site_map.add_child(marker)

#####
circle1 = folium.Circle(launchsites_lists[1], radius=1000, color='red', fill = True).add_child(folium.Popup('CCAFS SLC-40'))
marker1 = folium.Marker(launchsites_lists[1], icon=DivIcon(icon_size=(20,20),icon_anchor=(0,0), html='<div style="font-size: 12; color:red"><b>X</b></div>'),
site_map.add_child(circle1)
site_map.add_child(marker1)

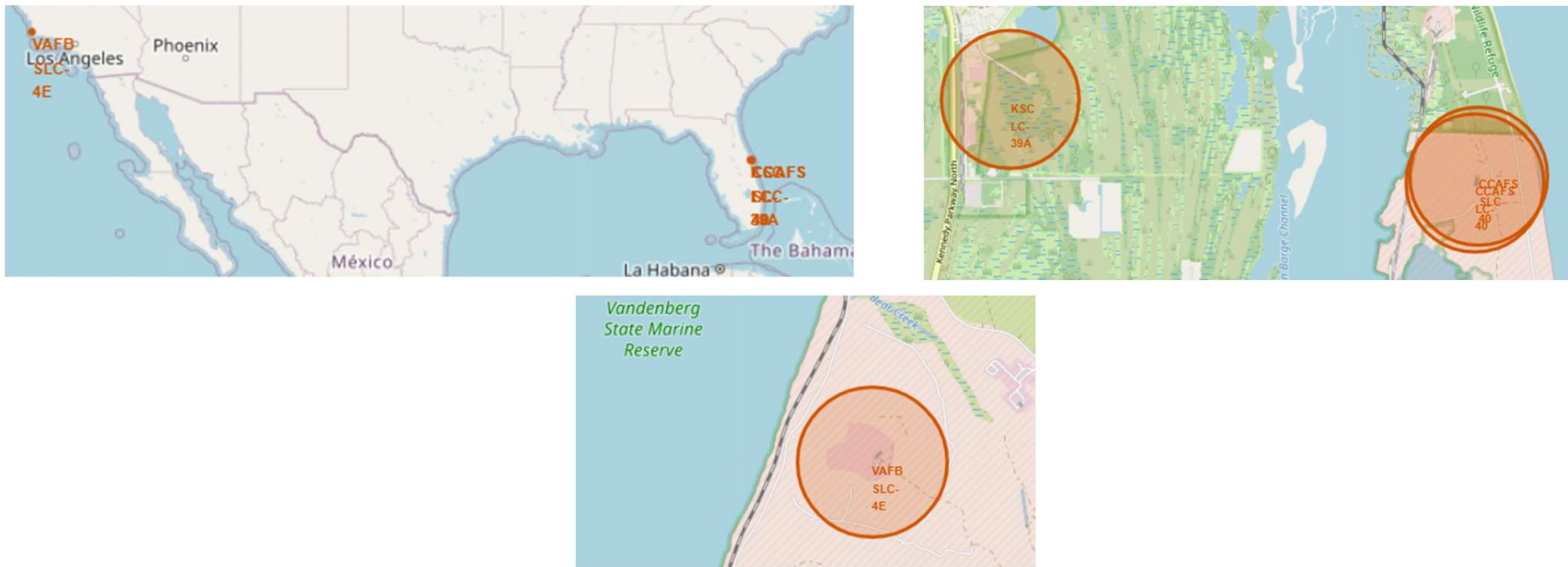
#####
circle2 = folium.Circle(launchsites_lists[2], radius=1000, color='red', fill = True).add_child(folium.Popup('KSC SLC-39A'))
marker2 = folium.Marker(launchsites_lists[2], icon=DivIcon(icon_size=(20,20),icon_anchor=(0,0), html='<div style="font-size: 12; color:red"><b>X</b></div>'),
site_map.add_child(circle2)
site_map.add_child(marker2)

#####
circle3 = folium.Circle(launchsites_lists[3], radius=1000, color='red', fill = True).add_child(folium.Popup('VAFB SLC-4E'))
marker3 = folium.Marker(launchsites_lists[3], icon=DivIcon(icon_size=(20,20),icon_anchor=(0,0), html='<div style="font-size: 12; color:red"><b>X</b></div>'),
site_map.add_child(circle3)
site_map.add_child(marker3)
```

EDA and interactive visual analytics methodology

Folium : Step 4 Continued

- Shown below is the outcome of adding the following launch site coordinates into the map



- As shown in the image, most of the launch sites are close to the coast. The only launch site that is not near the coast line is KSC LC-39A, however, it is still near water like the rest of them

EDA and interactive visual analytics methodology

Folium : Step 5

- After finding the location of each launch site, I next wanted to see the success rate of first stage landing of each launch site.
- Therefore, I created a markercluster object since many launch records have the exact same coordinate and keep the map simplified. The markercluster will be based off the class column in the data frame
 - If the class=1 then the marker is green, if the class=0 then the marker is red
- In order for the marker to change color, I created a function called marker_color

In [13]:

```
import folium
from folium.plugins import MarkerCluster

# Step 1: Create the map object with a unique name
site_map = folium.Map(location=[28.5634, -80.5768], zoom_start=10) # Example coordinates

# Step 2: Create the MarkerCluster object
marker_cluster = MarkerCluster()

# Step 3: Add the MarkerCluster to the map
marker_cluster.add_to(site_map)
```

In [14]:

```
# Apply a function to check the value of `class` column
# If class=1, marker_color value will be green
# If class=0, marker_color value will be red
color_list = []
def marker_color(value):
    marker_color_list = []
    for i in value:
        if i == 1:
            marker_color = 'green'
            marker_color_list.append(marker_color)
        elif i == 0:
            marker_color = 'red'
            marker_color_list.append(marker_color)
        else:
            print('no color available')
    color_list.append(marker_color_list)
    return color_list
marker_color(spacex_df['class'])

spacex_df['marker_color']=color_list[0]
print(spacex_df.head())
```

	Launch Site	Lat	Long	class	marker_color
0	CCAFS LC-40	28.562302	-80.577356	0	red
1	CCAFS LC-40	28.562302	-80.577356	0	red
2	CCAFS LC-40	28.562302	-80.577356	0	red
3	CCAFS LC-40	28.562302	-80.577356	0	red
4	CCAFS LC-40	28.562302	-80.577356	0	red

EDA and interactive visual analytics methodology

Folium : Step 5 Continued

- As shown in the Interactive map with Folium Task 2 results slide, majority of the launches were in Florida specially at the CCAFS LC-40 launch site
- However, most of the launches from CCAFS LC-40 were unsuccessful compared to the rest of the launch sites
- The launch site that had the most successful was KSC LC-39A

EDA and interactive visual analytics methodology

Folium : Step 6

- After visualizing the success rate at the launch sites, I would to see what were the proximities of the nearest highway, coastline, railroad, and city from the launch site
- In order to find these distances I created a function to calculate the distance using lat and lon coordinates
- I also used the mouse position function from the Folium library in order to get those coordinates from the nearest highway, coastline, railroad, and city from the launch site
- And, the polyline function from the Folium library to create a distance line from the launch site to those proximities

```
In [17]: from math import sin, cos, sqrt, atan2, radians

def calculate_distance(lat1, lon1, lat2, lon2):
    # approximate radius of earth in km
    R = 6373.0

    lat1 = radians(lat1)
    lon1 = radians(lon1)
    lat2 = radians(lat2)
    lon2 = radians(lon2)

    dlon = lon2 - lon1
    dlat = lat2 - lat1

    a = sin(dlat / 2)**2 + cos(lat1) * cos(lat2) * sin(dlon / 2)**2
    c = 2 * atan2(sqrt(a), sqrt(1 - a))

    distance = R * c
    return distance
```

```
In [ ]: from folium.plugins import MousePosition

# Create the map object
site_map = folium.Map(location=[28.5634, -80.5768], zoom_start=10) # Example coordinates

# Add MousePosition plugin to the map
mouse_position = MousePosition()
mouse_position.add_to(site_map)

# Display the map with MousePosition plugin
site_map
```

```
In [ ]: # Create a 'folium.PolyLine' object using the coastline coordinates and launch site coordinate
# lines=folium.PolyLine(locations=coordinates, weight=1)
folium.PolyLine(
    locations=[[launch_site_lat, launch_site_lon], [coastline_lat, coastline_lon]],
    color='red'
).add_to(site_map)

# Display the map with the PolyLine
site_map
```

EDA and interactive visual analytics methodology

Folium : Step 6 Continued

- The results after running the previous lines of code are shown in EDA and interactive visual analytics Task 3 Results slide.
- But, the patterns I noticed is that the launch sites try
 - to stay at least 15km away from cities
 - to stay as close to the coastline, highway, and railroad as possible
 - The goal is to be roughly 2km away from both the railroad and coastline while at least 6km away from the highway

EDA and interactive visual analytics methodology

Plotly

- After finding the proximities from the launch site, I still needed to have an answer to the following questions below
 1. Which site has the largest successful launches?
 2. Which site has the highest launch success rate?
 3. Which payload range(s) has the highest launch success rate?
 4. Which payload range(s) has the lowest launch success rate?
 5. Which F9 Booster version (v1.0, v1.1, FT, B4, B5, etc.) has the highest?
- And, I knew that if I could create a dashboard application by using Plotly, I could answer though questions in real time.
- The following slides are how I created a dashboard application in order to answers though questions
- If you are interested in the outcomes of this data analysis, please skip ahead to the Plotly Dash dashboard results

EDA and interactive visual analytics methodology

Plotly : Step 1

- Like the folium data analysis,
 - I imported the following libraries from Python
 - And converted the csv file into a data frame

```
8     # Import required libraries
9     import pandas as pd
10    import dash
11    import dash_html_components as html
12    import dash_core_components as dcc
13    from dash.dependencies import Input, Output
14    import plotly.express as px
15
```

```
spacex_df = pd.read_csv("spacex_launch_dash.csv")
```

EDA and interactive visual analytics methodology

Plotly : Step 2

- Next, I created a drop down input component in order to select from different launch sites

```
20  spacex_options = [  
21      {'label': 'All Sites', 'value': 'ALL'},  
22      {'label': 'CCAFS LC-40', 'value': 'CCAFS LC-40'},  
23      {'label': 'VAFB SLC-4E', 'value': 'VAFB SLC-4E'},  
24      {'label': 'KSC LC-39A', 'value': 'KSC LC-39A'},  
25      {'label': 'CCAFS SLC-40', 'value': 'CCAFS SLC-40'}  
26  ]
```

```
# Create an app layout  
app.layout = html.Div(children=[html.H1('SpaceX Launch Records Dashboard',  
                                     style={'textAlign': 'center', 'color': '#503D36',  
                                             'font-size': 40}),  
                               # TASK 1: Add a dropdown list to enable Launch Site selection  
                               # The default select value is for ALL sites  
                               # dcc.Dropdown(id='site-dropdown',...)  
                               html.Div(dcc.Dropdown(id='site-dropdown',  
                                                       options=spacex_options,  
                                                       value='ALL',  
                                                       placeholder='Select a Launch Site here',  
                                                       searchable=True)),  
                               html.Br(),
```

EDA and interactive visual analytics methodology

Plotly : Step 3

- After creating the drop-down input component, I created a call back function to produce a pie chart based on the number of successful landings from the selected launch site

```
# TASK 2: Add a pie chart to show the total successful launches count for all sites
# If a specific launch site was selected, show the Success vs. Failed counts for the site
html.Div(dcc.Graph(id='success-pie-chart')),
html.Br(),

html.P("Payload range (Kg):"),
```

```
72 # TASK 2:
73 # Add a callback function for `site-dropdown` as input, `success-pie-chart` as output
74 @app.callback(Output(component_id='success-pie-chart', component_property='figure'),
75               Input(component_id='site-dropdown', component_property='value'))
76 def get_pie_chart(entered_site):
77     filtered_df = spacex_df
78     if entered_site == 'ALL':
79         fig = px.pie(filtered_df, values='class',
80                     names='Launch Site',
81                     title='Total Success Launches by All Sites')
82         return fig
83     else:
84         filtered_df = spacex_df[spacex_df['Launch Site'] == entered_site]
85         fig = px.pie(filtered_df, names='class', title=f'Total Success Launches for site {entered_site}')
86         return fig
87     # return the outcomes piechart for a selected site
```

EDA and interactive visual analytics methodology

Plotly : Step 4

- Once the call back pie chart function was created, I next added a ranger slider when selecting a payload since the payload is correlated to the mission outcome.

```
# TASK 3: Add a slider to select payload range
#dcc.RangeSlider(id='payload-slider',...)
html.Div(dcc.RangeSlider(id='payload-slider',
                          min=0,max=10000,step=1000,
                          marks=spacex_marks,
                          value=[min_payload,max_payload])),

# TASK 4: Add a scatter chart to show the correlation between payload and launch success
html.Div(dcc.Graph(id='success-payload-scatter-chart')),
1)
```

```
27  ▼ spacex_marks={0: '0',
28                      1000: '1000',
29                      2000: '2000',
30                      3000: '3000',
31                      4000: '4000',
32                      5000: '5000',
33                      6000: '6000',
34                      7000: '7000',
35                      8000: '8000',
36                      9000: '9000',
37                      10000: '10000'}
```


EDA and interactive visual analytics methodology

Plotly : Step 5

- Next, I created another call back function to produce a scatter plot based on the payload and launch site.

```
# TASK 4: Add a scatter chart to show the correlation between payload and launch success
html.Div(dcc.Graph(id='success-payload-scatter-chart'),
)
```

```
88 # TASK 4:
89 # Add a callback function for `site-dropdown` and `payload-slider` as inputs, `success-payload-scatter-chart` as output
90 @app.callback(Output(component_id='success-payload-scatter-chart', component_property='figure'),
91               [Input(component_id='site-dropdown', component_property='value'),
92                Input(component_id='payload-slider', component_property='value')])
93 def get_scatter_plot(entered_site, entered_payload):
94     min_payload, max_payload = entered_payload
95     filtered_df = spacex_df[(spacex_df['Payload Mass (kg)'] >= min_payload) & (spacex_df['Payload Mass (kg)'] <= max_payload)]
96     if entered_site != 'ALL':
97         filtered_df = filtered_df[filtered_df['Launch Site'] == entered_site]
98         fig = px.scatter(filtered_df, x='Payload Mass (kg)', y='class', color='Booster Version Category', title=f'Success by Payload Mass for {entered_site}')
99         return fig
100     else:
101         entered_site == 'ALL'
102         fig = px.scatter(filtered_df, x='Payload Mass (kg)', y='class', color='Booster Version Category', title='Success by Payload Mass for ALL')
103         return fig
```

EDA and interactive visual analytics methodology

Plotly : Step 6

- After creating the scatter plot call back function, I ran the code and was able to find out the answers to the following questions that were mentioned previously
 - KSC LC-39A was the largest successful launch and highest successful launch rate
 - The payload range from 2kg to 4kg was the highest, while 2kg to 4kg and 4kg to 6kg were the lowest
 - And, F9 Booster version FT had the highest launch success rate

Plotly Dash dashboard results

- The following slides are plotly dash results from:
 - Plotly Dash Lab Tasks 1 through 4
 - Finding Insights Visually Questions:
 1. Which site has the largest successful launches?
 2. Which site has the highest launch success rate?
 3. Which payload range(s) has the highest launch success rate?
 4. Which payload range(s) has the lowest launch success rate?
 5. Which F9 Booster version (v1.0, v1.1, FT, B4, B5, etc.) has the highest?

predictive analysis methodology

- In order to determine if the first stage will land from the data I obtain from the Data collection and Data wrangling slides, I created a machine learning pipeline in order to predict if the outcome of the first stage.
- The following slides are the steps on how I performed my predictive analysis

predictive analysis methodology Step 1

- First, I imported the following libraries into my python code

```
In [2]: # Pandas is a software library written for the Python programming language for data manipulation
import pandas as pd
# NumPy is a library for the Python programming language, adding support for large, multi-dimensional
import numpy as np
# Matplotlib is a plotting library for python and pyplot gives us a MatLab like plotting framework
import matplotlib.pyplot as plt
#Seaborn is a Python data visualization library based on matplotlib. It provides a high-level interface
import seaborn as sns
# Preprocessing allows us to standardize our data
from sklearn import preprocessing
# Allows us to split our data into training and testing data
from sklearn.model_selection import train_test_split
# Allows us to test parameters of classification algorithms and find the best one
from sklearn.model_selection import GridSearchCV
# Logistic Regression classification algorithm
from sklearn.linear_model import LogisticRegression
# Support Vector Machine classification algorithm
from sklearn.svm import SVC
# Decision Tree classification algorithm
from sklearn.tree import DecisionTreeClassifier
# K Nearest Neighbors classification algorithm
from sklearn.neighbors import KNeighborsClassifier
```

predictive analysis methodology Step 1 Continued

- Next, I loaded in data (csv files from the data collection and wrangling slides) and converted them into data frames
- Note:
 - URL1 is csv file from data_falcon9 data frame
 - URL2 is the csv file from features_ones_hot data frame

```
[65]: from js import fetch
import io

URL1 = "https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/IBM-DS0321EN-SkillsNetwork/datasets/dataset_part_2.csv"
resp1 = await fetch(URL1)
text1 = io.BytesIO((await resp1.arrayBuffer()).to_py())
data = pd.read_csv(text1)
```

```
[67]: URL2 = 'https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/IBM-DS0321EN-SkillsNetwork/datasets/dataset_part_3.csv'
resp2 = await fetch(URL2)
text2 = io.BytesIO((await resp2.arrayBuffer()).to_py())
X = pd.read_csv(text2)
```

- Note:
 - Data contains the Target value which is class while X contains the independent variables that affect the Target value

predictive analysis methodology Step 2

- Next, I converted the data frame column Class into a numpy array since that is the target value we are interested in. As shown in the image below

```
[69]: print('task 1')
      print('getting Class col from data ')
      print('Class is now  in a panda series called Y')
      print()
      Y=data['Class'].to_numpy()
      Y=pd.Series(Y)
      print('end of task 1')
      print()

task 1
getting Class col from data
Class is now  in a panda series called Y

end of task 1
```

predictive analysis methodology Step 3

- After getting out target data and converting into an array, I need to standardize the data in X by using the functions shown below

```
[70]: # students get this
      #transform = preprocessing.StandardScaler()
      from sklearn.preprocessing import StandardScaler
      print('task 2')
      X = preprocessing.StandardScaler().fit(X).transform(X)
      print('Standardize the data in X')
      print('end of task 2')

      task 2
      Standardize the data in X
      end of task 2
```


predictive analysis methodology Step 3

- Now that X is standardized and Y is an array, I split the data into a training set and a testing set by using the function below

```
[71]: X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size=0.2, random_state=2)
      print('split test and training data')
      split test and training data
```

- As shown in the image below, there are only 18 test samples for Y_test

```
[72]: Y_test.shape
[72]: (18,)
```

predictive analysis methodology Step 4

- After splitting my data into testing and training, I needed to create a model that would predict the first stage of landing, however, I was not sure which one would be best so I tried several different models as shown in the following slides.
- I also created a function to display a confusion plot in order to get a visualization of the model as shown in the image below

```
[3]: def plot_confusion_matrix(y,y_predict):  
      "this function plots the confusion matrix"  
      from sklearn.metrics import confusion_matrix  
  
      cm = confusion_matrix(y, y_predict)  
      ax= plt.subplot()  
      sns.heatmap(cm, annot=True, ax = ax); #annot=True to annotate cells  
      ax.set_xlabel('Predicted labels')  
      ax.set_ylabel('True labels')  
      ax.set_title('Confusion Matrix');  
      ax.xaxis.set_ticklabels(['did not land', 'land']); ax.yaxis.set_ticklabels(['did not land', 'landed'])  
      plt.show()
```

predictive analysis methodology Step 4 Continued

- The first model that I used was the logistic regression model. After I created an log regression object, I created a GridSearchCV object in order to find the best parameters for the log regression model as shown in the image below.

```
[90]: parameters = {'C':[0.01,0.1,1],  
                  'penalty':['l2'],  
                  'solver':['lbfgs']  
[27]: print("tuned hpyerparameters :(best parameters) ",logreg_cv.best_params_)  
      print("accuracy :",logreg_cv.best_score_)  
      print("Best estimator: ", logreg_cv.best_estimator_)  
  
tuned hpyerparameters :(best parameters) {'C': 0.01, 'penalty': 'l2', 'solver': 'lbfgs'}  
accuracy : 0.8464285714285713  
Best estimator: LogisticRegression(C=0.01)  
  
scoring='accuracy')  
  estimator: LogisticRegression  
LogisticRegression()  
  LogisticRegression
```

- Next, I train the model with the training data (X_train,Y_train) by using the fit function as shown above
- After training the logistic regression model, I printed out the best score, best parameter, and best estimator for the logistic regression model as shown below

predictive analysis methodology Step 4 Continued

- After training the logistic regression model, I printed out the best score, best parameter, and best estimator for the logistic regression model and calculated the accuracy of the model by using the score method as shown below

```
[27]: print("tuned hpyerparameters :(best parameters) ",logreg_cv.best_params_)
      print("accuracy :",logreg_cv.best_score_)
      print("Best estimator: ", logreg_cv.best_estimator_)

tuned hpyerparameters :(best parameters) {'C': 0.01, 'penalty': 'l2', 'solver': 'lbfgs'}
accuracy : 0.8464285714285713
Best estimator: LogisticRegression(C=0.01)
```

```
[42]: #yhat=logreg_cv.predict(X_test)
      from sklearn.metrics import classification_report
      yhat = logreg_cv.best_estimator_.predict(X_test)
      from sklearn.metrics import accuracy_score
      accuracy = accuracy_score(Y_test, yhat)
      print(classification_report(Y_test, yhat))
      print("Accuracy log reg:", accuracy)
      plot_confusion_matrix(Y_test,yhat)

              precision    recall  f1-score   support

    0       1.00      0.50      0.67         6
    1       0.80      1.00      0.89        12

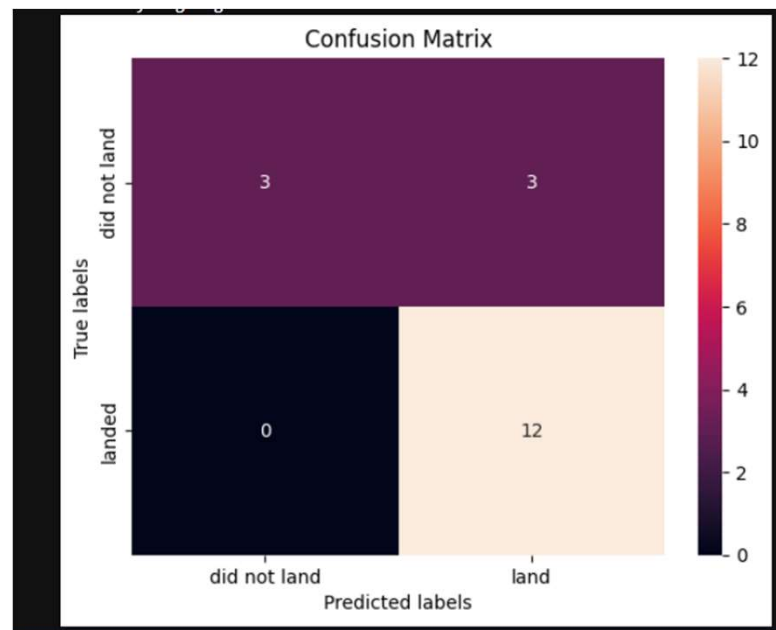
 accuracy
macro avg      0.90      0.75      0.78        18
weighted avg   0.87      0.83      0.81        18

Accuracy log reg: 0.8333333333333334
```

- As shown in the image the accuracy of the logistic regression model is 83%
- Note: the reason why I used the best estimator to predict the test data is because the best estimator helps pick the best parameters for the model.

predictive analysis methodology Step 4 Continued

- Here is an image of the confusion plot for the logistic regression model



- As shown in the confusion plot,
 - logistic regression can distinguish between the different classes.
 - However there is a major problem with the False positives

predictive analysis methodology Step 4 Continued

- The next model I tried was the SVM (Support Vector Machine) model
- I performed the same following steps as mentioned in the logistic regression model
- Like logistic regression, I created an SVM object and then used gridsearchCV to get the best parameters for that object.
- Also like logistic regression, I printed out the best score, best parameters, and best estimator

```
[20]: parameters = {'kernel':('linear', 'rbf','poly','rbf', 'sigmoid'),
                  'C': np.logspace(-3, 3, 5),
                  'gamma':np.logspace(-3, 3, 5)}
      svm = SVC()

[22]: svm_cv = GridSearchCV(svm, parameters, cv=10, error_score='raise')
      svm_cv.fit(X_train, Y_train)

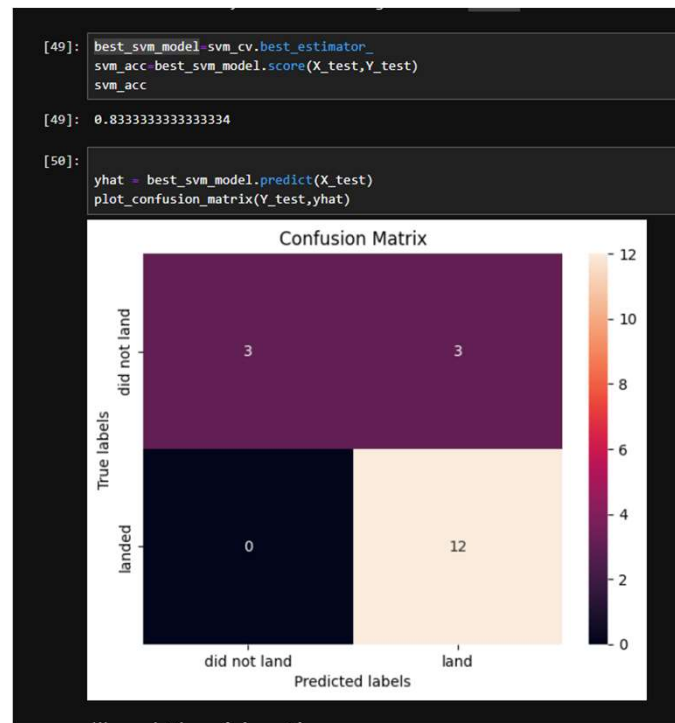
[22]: > GridSearchCV ⓘ ⓘ
      > estimator: SVC
      > SVC ⓘ

[25]: print("tuned hyperparameters :(best parameters) ",svm_cv.best_params_)
      print("accuracy :",svm_cv.best_score_)
      print("Best estimator: ", svm_cv.best_estimator_)

tuned hyperparameters :(best parameters) {'C': 1.0, 'gamma': 0.03162277660168379, 'kernel': 'sigmoid'}
accuracy : 0.8482142857142856
Best estimator: SVC(gamma=0.03162277660168379, kernel='sigmoid')
```

predictive analysis methodology Step 4 Continued

- And just like the Logistic Regression model, I calculated the accuracy of the model
- The model of the accuracy is 83%



predictive analysis methodology Step 4 Continued

- The next model I tried was the tree decision model
- And just like the other two models, I preformed the same following steps

```
[51]: parameters = {'criterion': ['gini', 'entropy'],
                  'splitter': ['best', 'random'],
                  'max_depth': [2*n for n in range(1,10)],
                  'max_features': ['log2', 'sqrt'],
                  'min_samples_leaf': [1, 2, 4],
                  'min_samples_split': [2, 5, 10]}

tree = DecisionTreeClassifier()

[52]: tree_cv = GridSearchCV(tree, parameters, cv=10, error_score='raise')
tree_cv.fit(X_train, Y_train)

[52]: > GridSearchCV ① ②
      > estimator: DecisionTreeClassifier
          > DecisionTreeClassifier ③

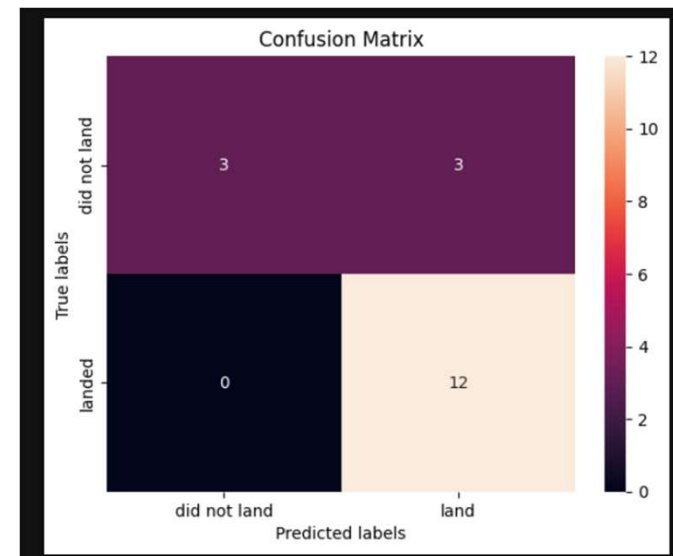
[33]: print("tuned hpyerparameters :(best parameters) ",tree_cv.best_params_)
print("accuracy :",tree_cv.best_score_)
print("Best estimator: ", tree_cv.best_estimator_)

tuned hpyerparameters :(best parameters) {'criterion': 'gini', 'max_depth': 8, 'max_features': 'sqrt', 'min_samples_leaf': 1, 'min_samples_split': 5, 'splitter': 'random'}
accuracy : 0.8910714285714286
Best estimator: DecisionTreeClassifier(max_depth=8, max_features='sqrt', min_samples_split=5,
splitter='random')
```


predictive analysis methodology Step 4 Continued

- The accuracy of the tree decision model also turned out to 83% and the image of the confusion plot was the same as the other models
- I started to become worried since all three models were the same but I checked my code and everything seemed right

```
[53]: tree_best_model=tree_cv.best_estimator_  
      tree_acc=tree_best_model.score(X_test,Y_test)  
      tree_acc  
[53]: 0.8333333333333334
```



predictive analysis methodology Step 4 Continued

- The last model I tried was the K nearest Neighbor model, and just like the last three models I preformed the same steps as the rest

```
[36]: parameters = {'n_neighbors': [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
                  'algorithm': ['auto', 'ball_tree', 'kd_tree', 'brute'],
                  'p': [1,2]}

      KNN = KNeighborsClassifier()

[37]: knn_cv = GridSearchCV(KNN, parameters, cv=10, error_score='raise')
      knn_cv.fit(X_train, Y_train)

[37]: > GridSearchCV ⓘ ?
      > estimator: KNeighborsClassifier
      > KNeighborsClassifier ⓘ

[38]: print("tuned hyperparameters :(best parameters) ",knn_cv.best_params_)
      print("accuracy :",knn_cv.best_score_)
      print("Best estimator: ", knn_cv.best_estimator_)

      tuned hyperparameters :(best parameters) {'algorithm': 'auto', 'n_neighbors': 10, 'p': 1}
      accuracy : 0.8482142857142858
      Best estimator: KNeighborsClassifier(n_neighbors=10, p=1)
```

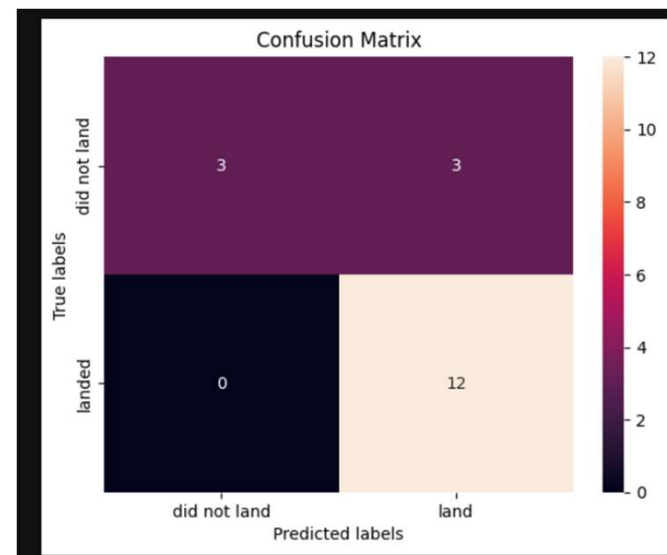
predictive analysis methodology Step 4 Continued

- And just like the rest of the models, the accuracy was 83% and the confusion plot was the same

```
[39]: from sklearn.metrics import classification_report
yhat = knn_cv.best_estimator_.predict(X_test)
from sklearn.metrics import accuracy_score
accuracy = accuracy_score(Y_test, yhat)
print(classification_report(Y_test, yhat))
print("Accuracy KNN:", accuracy)
plot_confusion_matrix(Y_test, yhat)
```

	precision	recall	f1-score	support
0	1.00	0.50	0.67	6
1	0.80	1.00	0.89	12
accuracy			0.83	18
macro avg	0.90	0.75	0.78	18
weighted avg	0.87	0.83	0.81	18

Accuracy KNN: 0.8333333333333334



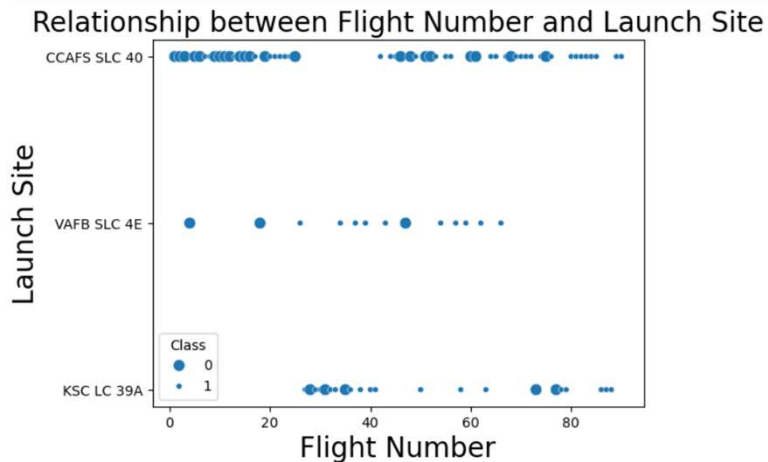
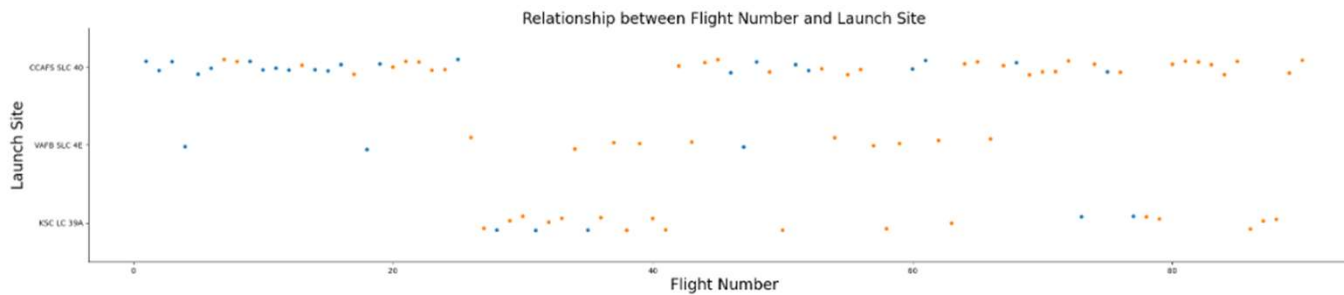
- To be honest, I am not sure what could have happened I double checked everything and every time I ran the code it would be the same outcome. Therefore, I am not really sure what model performs the best. However, if I were to decide what model to use I would use the Logistic Regression model. Sorry for the inconvenience

EDA with visualization results

- The following slides are results from:
 - Visualization Lab Tasks 1 through 8

Visualization Task 1 Results

• TASK 1: Visualize the relationship between Flight Number and Launch Site

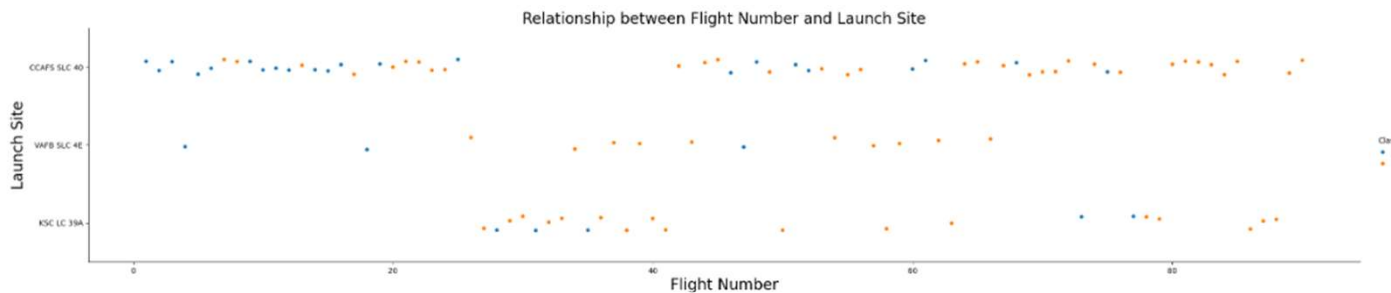


Finding Patterns in both Plots:

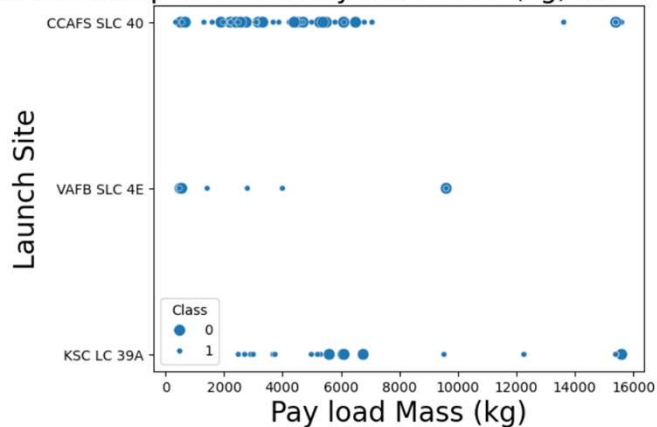
- As shown in the plots, you can see that different launch sites have different success rates.
- According to both plots, launch site CCAFS SLC-40 has the lowest successful launch rate.
 - The reason why it is so low is because majority of flight numbers 1 through 20 were unsuccessful
- According to both plots, launch sites VAFB SLC – 4E and KSC LC -39A have a large amount of successful launches between flight numbers 50 through 70 while launch site CCAFS SLC – 40 has a large amount of successful launches after flight number 80

Visualization Task 2 Results

• TASK 2: Visualize the relationship between Payload and Launch Site



Relationship between Pay Load Mass (kg) and Launch Site



Finding Patterns in both Plots:

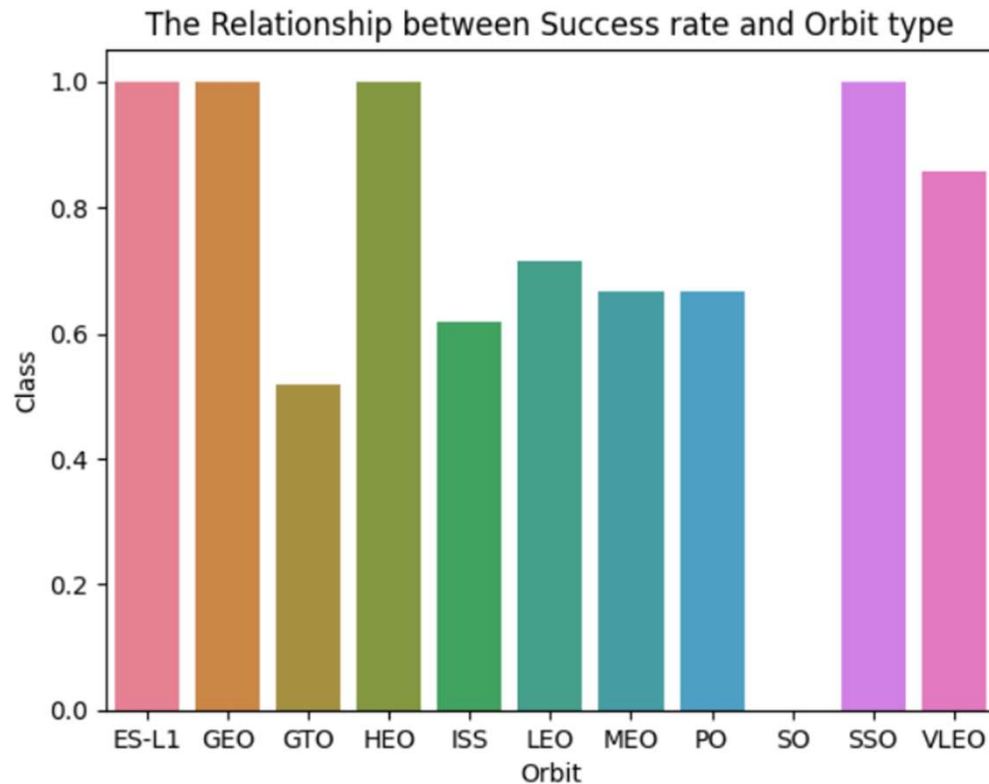
- As shown in the plots, you can see that VAFB SLC – 4E had no rocket launches with a heavy payload mass greater than 10,000kg
- As shown in the plots, majority of the unsuccessful launches at CCAFS SLC – 40 had a payload ranging from 1000 kg to 6000 kg roughly

Visualization Task 3 Results

• TASK 3: Visualize the relationship between Success rate of each Orbit type

Finding Which Orbits have high Success rate:

- As shown in the plot, ES-L1, GEO, HEO, SSO have the highest success rate
 - While SO has the lowest success rate at 0%

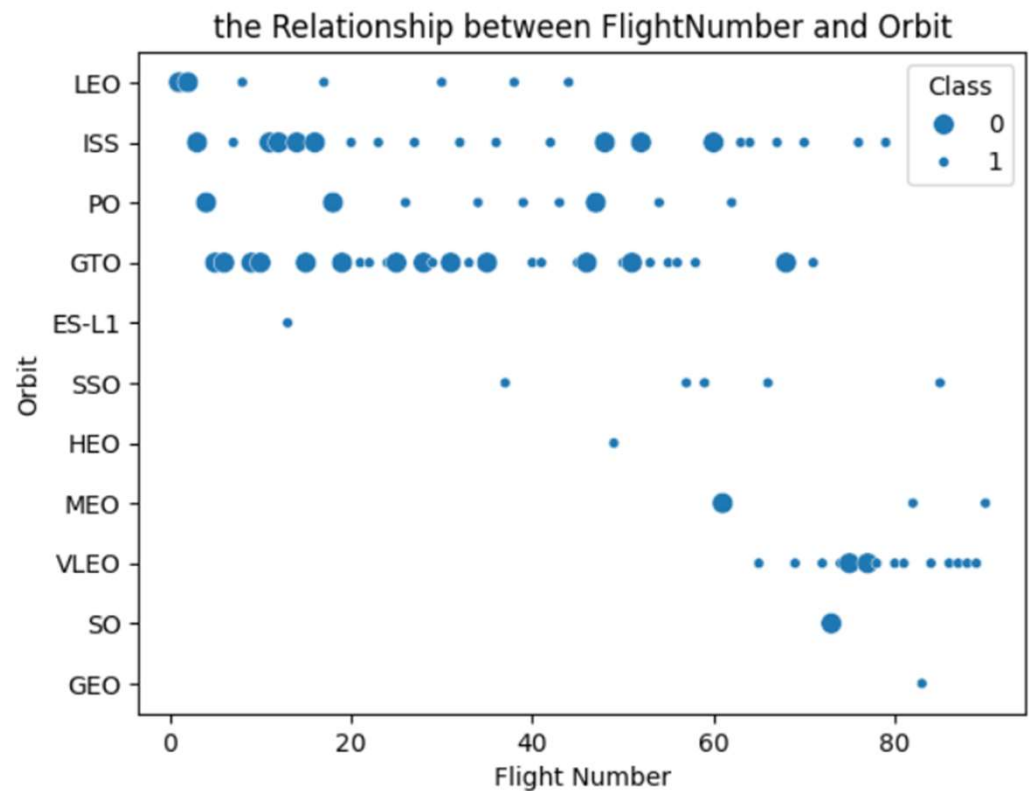


Visualization Task 4 Results

- TASK 4: Visualize the relationship between Flight number and Orbit type

Finding Relationship Between Flight number and Orbit Type:

- As shown in the plots, the LEO orbit success appears related to the number of flights, however, when in GTO orbit, there is no relationship between flight number

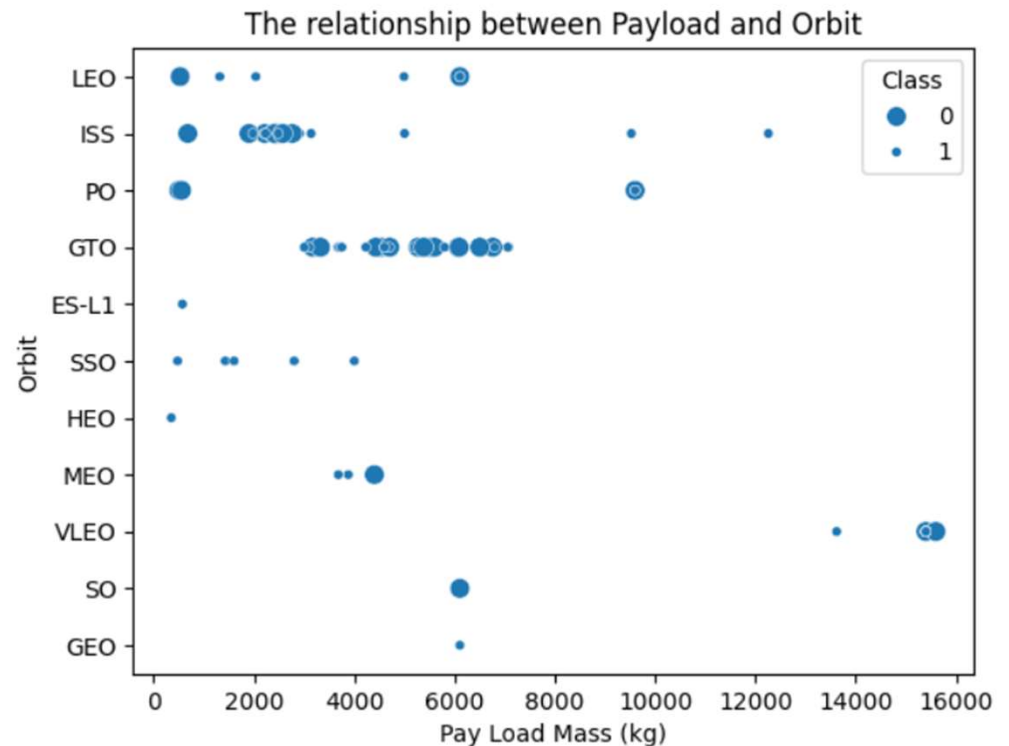


Visualization Task 5 Results

• TASK 5: Visualize the relationship between Payload and Orbit type

Finding Relationship Between Payload and Orbit Type:

- As shown in the plot, with heavy payloads the successful landing or positive landing rate are more for PO, LEO, and ISS
- However, for GTO you cannot distinguish this well as both positive landing rate and negative landing

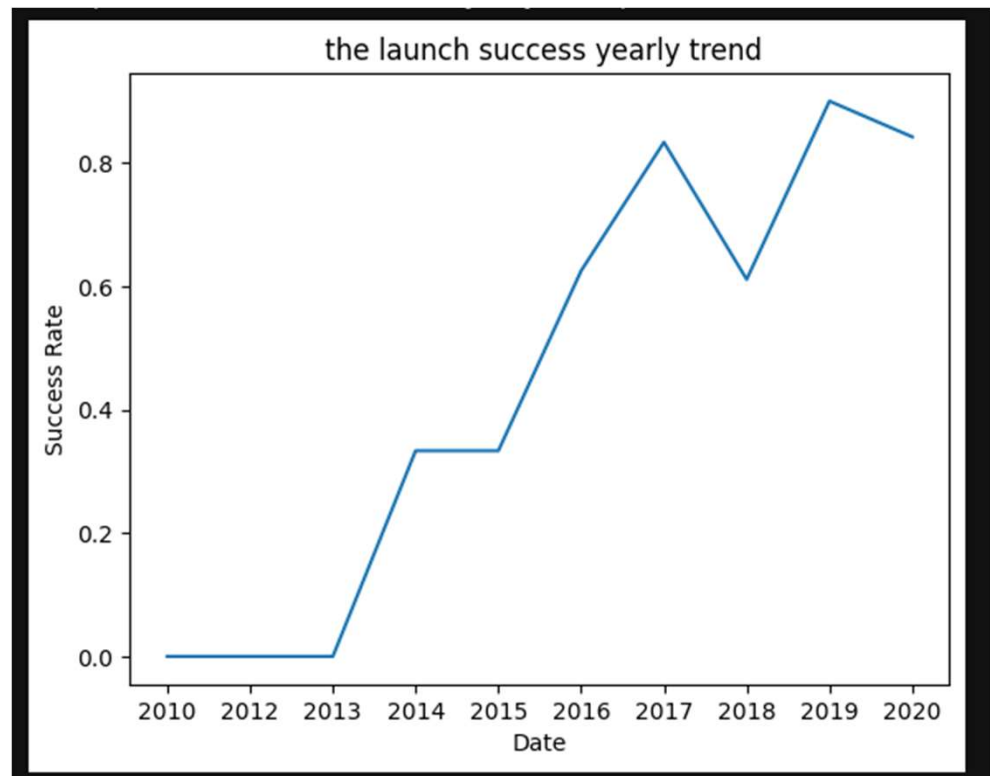


Visualization Task 6 Results

- TASK 6: Visualize the launch success yearly trend

Finding the Launch Success Yearly Trend:

- As shown in the line plot, the success rate has been increasing since 2013



Visualization Task 7 Results

- TASK 7: Create dummy variables to categorical columns

```
[9]: # HINT: Use get_dummies() function on the categorical columns
cols = ['Orbit', 'LaunchSite', 'LandingPad', 'Serial']
features_one_hot = pd.get_dummies(features, columns=cols)
features_one_hot.head()
```

```
[9]:
```

	FlightNumber	PayloadMass	Flights	GridFins	Reused	Legs	Block	ReusedCount	Orbit_ES-L1	Orbit_GEO	...	Serial_B1048	Serial_B1049	Serial_B1050	Serial_B1051	Serial_B1054	Serial_B1056	Serial_B1058
0	1	6104.959412	1	False	False	False	1.0	0	0	0	...	0	0	0	0	0	0	0
1	2	525.000000	1	False	False	False	1.0	0	0	0	...	0	0	0	0	0	0	0
2	3	677.000000	1	False	False	False	1.0	0	0	0	...	0	0	0	0	0	0	0
3	4	500.000000	1	False	False	False	1.0	0	0	0	...	0	0	0	0	0	0	0
4	5	3170.000000	1	False	False	False	1.0	0	0	0	...	0	0	0	0	0	0	0

5 rows x 80 columns

Visualization Task 8 Results

- TASK 8: Cast all numeric columns to 'float64'

```
features_one_hot.astype('float64')
print(type(features_one_hot))
features_one_hot
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
[10]:
```

	FlightNumber	PayloadMass	Flights	GridFins	Reused	Legs	Block	ReusedCount	Orbit_ES-L1	Orbit_GEO	...	Serial_B1048	Serial_B1049	Serial_B1050	Serial_B1051	Serial_B1054	Serial_B1056	Serial_B1058	S
0	1	6104.959412	1	False	False	False	1.0	0	0	0	...	0	0	0	0	0	0	0	
1	2	525.000000	1	False	False	False	1.0	0	0	0	...	0	0	0	0	0	0	0	
2	3	677.000000	1	False	False	False	1.0	0	0	0	...	0	0	0	0	0	0	0	
3	4	500.000000	1	False	False	False	1.0	0	0	0	...	0	0	0	0	0	0	0	
4	5	3170.000000	1	False	False	False	1.0	0	0	0	...	0	0	0	0	0	0	0	
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
85	86	15400.000000	2	True	True	True	5.0	2	0	0	...	0	0	0	0	0	0	0	
86	87	15400.000000	3	True	True	True	5.0	2	0	0	...	0	0	0	0	0	0	0	1
87	88	15400.000000	6	True	True	True	5.0	5	0	0	...	0	0	0	1	0	0	0	
88	89	15400.000000	3	True	True	True	5.0	2	0	0	...	0	0	0	0	0	0	0	
89	90	3681.000000	1	True	False	True	5.0	0	0	0	...	0	0	0	0	0	0	0	

90 rows x 80 columns

EDA with SQL results slides

- The following slides are EDA with SQL results:
 - Tasks 1 through 10

EDA with SQL Task 1 results

- Task 1 Display the names of the unique launch sites in the space mission

```
In [9]: %sql select DISTINCT(Launch_Site) from SPACEXTABLE
```

```
* sqlite:///my_data1.db
```

```
Done.
```

```
Out[9]: Launch_Site
```

```
CCAFS LC-40
```

```
VAFB SLC-4E
```

```
KSC LC-39A
```

```
CCAFS SLC-40
```

EDA with SQL Task 2 results

- Task 2 Display 5 records where launch sites begin with the string 'CCA'

```
In [10]: %sql select* from SPACEXTABLE WHERE Launch_Site like 'CCA%' LIMIT 5
```

```
* sqlite:///my_data1.db  
Done.
```

Out[10]:

Date	Time (UTC)	Booster_Version	Launch_Site	Payload	PAYLOAD_MASS_KG_	Orbit	Customer	Mission_Outcome	Landing_Outcome
2010-06-04	18:45:00	F9 v1.0 B0003	CCAFS LC-40	Dragon Spacecraft Qualification Unit	0	LEO	SpaceX	Success	Failure (parachute)
2010-12-08	15:43:00	F9 v1.0 B0004	CCAFS LC-40	Dragon demo flight C1, two CubeSats, barrel of Brouere cheese	0	LEO (ISS)	NASA (COTS) NRO	Success	Failure (parachute)
2012-05-22	7:44:00	F9 v1.0 B0005	CCAFS LC-40	Dragon demo flight C2	525	LEO (ISS)	NASA (COTS)	Success	No attempt
2012-10-08	0:35:00	F9 v1.0 B0006	CCAFS LC-40	SpaceX CRS-1	500	LEO (ISS)	NASA (CRS)	Success	No attempt
2013-03-01	15:10:00	F9 v1.0 B0007	CCAFS LC-40	SpaceX CRS-2	677	LEO (ISS)	NASA (CRS)	Success	No attempt

EDA with SQL Task 3 results

- Task 3 Display the total payload mass carried by boosters launched by NASA (CRS)

```
In [11]: %sql select sum(PAYLOAD_MASS__KG_) as total_payload from SPACEXTABLE where Customer = 'NASA (CRS)'  
* sqlite:///my_data1.db  
Done.  
Out[11]: total_payload  
         45596
```


EDA with SQL Task 4 results

- Task 4 Display average payload mass carried by booster version F9 v1.1

```
In [12]: %sql select AVG(PAYLOAD_MASS_KG_) as avg_payload from SPACEXTABLE where Booster_Version = 'F9 v1.1'
* sqlite:///my_data1.db
Done.
Out[12]: avg_payload
         2928.4
```

EDA with SQL Task 5 results

- Task 5 List the date when the first successful landing outcome in ground was achieved

```
In [13]: %sql select MIN(Date) from SPACEXTABLE where Landing_Outcome = 'Success (ground pad)'
```

```
* sqlite:///my_data1.db  
Done.
```

```
Out[13]: MIN(Date)  
2015-12-22
```

EDA with SQL Task 6 results

- Task 6: List the names of the boosters which have success in drone ship and have a payload mass greater than 4000 but less 6000

```
[10]: %sql select Booster_Version from SPACEXTABLE where Landing_Outcome='Success (drone ship)' And PAYLOAD_MASS__KG_ between 4000 and 6000
```

```
* sqlite:///my_data1.db  
Done.
```

```
[10]: Booster_Version
```

```
F9 FT B1022
```

```
F9 FT B1026
```

```
F9 FT B1021.2
```

```
F9 FT B1031.2
```

EDA with SQL Task 7 results

- Task 7 List the total number of successful and failure mission outcomes

In [15]:

```
%sql select count(Landing_Outcome) from SPACEXTABLE
```

```
* sqlite:///my_data1.db
```

Done.

Out[15]:

count(Landing_Outcome)

101

EDA with SQL Task 8 results

- Task 8 List the names of the booster_versions which have carried the maximum payload mass. Use a subquery

```
In [16]: %sql select Booster_Version from SPACEXTABLE where PAYLOAD_MASS__KG_ = (select MAX(PAYLOAD_MASS__KG_) from SPACEXTABLE)
```

```
* sqlite:///my_data1.db
```

```
Done.
```

```
Out[16]: Booster_Version
```

```
F9 B5 B1048.4
```

```
F9 B5 B1049.4
```

```
F9 B5 B1051.3
```

```
F9 B5 B1056.4
```

```
F9 B5 B1048.5
```

```
F9 B5 B1051.4
```

```
F9 B5 B1049.5
```

```
F9 B5 B1060.2
```

```
F9 B5 B1058.3
```

```
F9 B5 B1051.6
```

```
F9 B5 B1060.3
```

```
F9 B5 B1049.7
```

EDA with SQL Task 9 results

- Task 9 List the records which will display the month names, failure landing_outcomes in drone ship ,booster versions, launch_site for the months in year 2015.

```
[11]: %sql SELECT substr(Date, 6, 2) as Month, Landing_Outcome, Booster_Version, Launch_Site FROM SPACEXTABLE WHERE substr(Date, 0, 5)='2015' AND Landing_Outcome='Failure (drone ship)'
```

* sqlite:///my_data1.db
Done.

```
[11]:
```

	Month	Landing_Outcome	Booster_Version	Launch_Site
01	Failure (drone ship)	F9 v1.1 B1012	CCAFS LC-40	
04	Failure (drone ship)	F9 v1.1 B1015	CCAFS LC-40	

EDA with SQL Task 10 results

- Task 10 Rank the count of landing outcomes (such as Failure (drone ship) or Success (ground pad)) between the date 2010-06-04 and 2017-03-20, in descending order.

```
[12]: %sql select Date,Landing_Outcome from SPACEXTABLE WHERE Landing_Outcome = 'Failure (drone ship)' or 'Success (ground pad)' and Date between '2010-06-04' and '2017-03-20' order by Date DESC
```

```
* sqlite:///my_data1.db  
Done.
```

```
[12]:
```

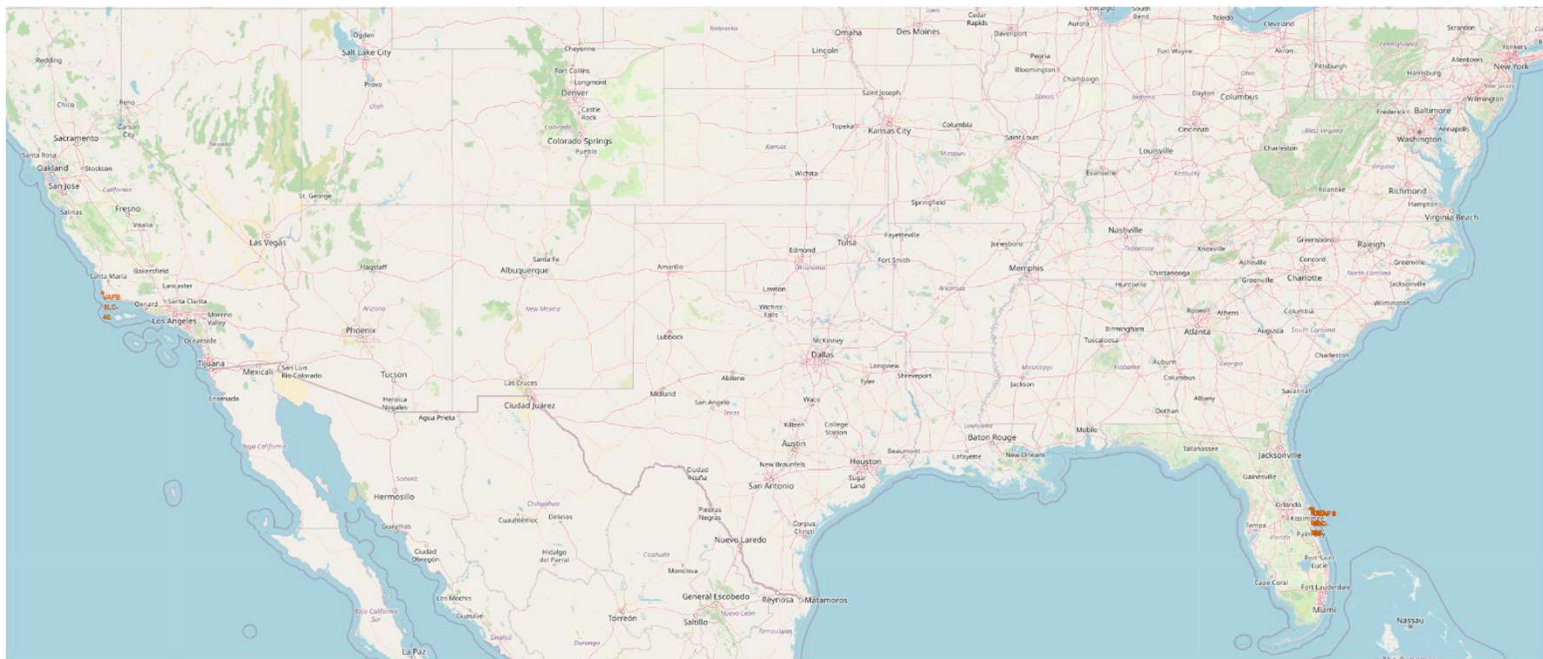
Date	Landing_Outcome
2016-06-15	Failure (drone ship)
2016-03-04	Failure (drone ship)
2016-01-17	Failure (drone ship)
2015-04-14	Failure (drone ship)
2015-01-10	Failure (drone ship)

Interactive map with Folium results

- The following slides are results from:
 - Folium Lab Tasks 1 through 3
 - Proximity Launch Site Questions:
 1. Are launch sites in close proximity to railways?
 2. Are launch sites in close proximity to highways?
 3. Are launch sites in close proximity to coastline?
 4. Do launch sites keep certain distance away from cities?

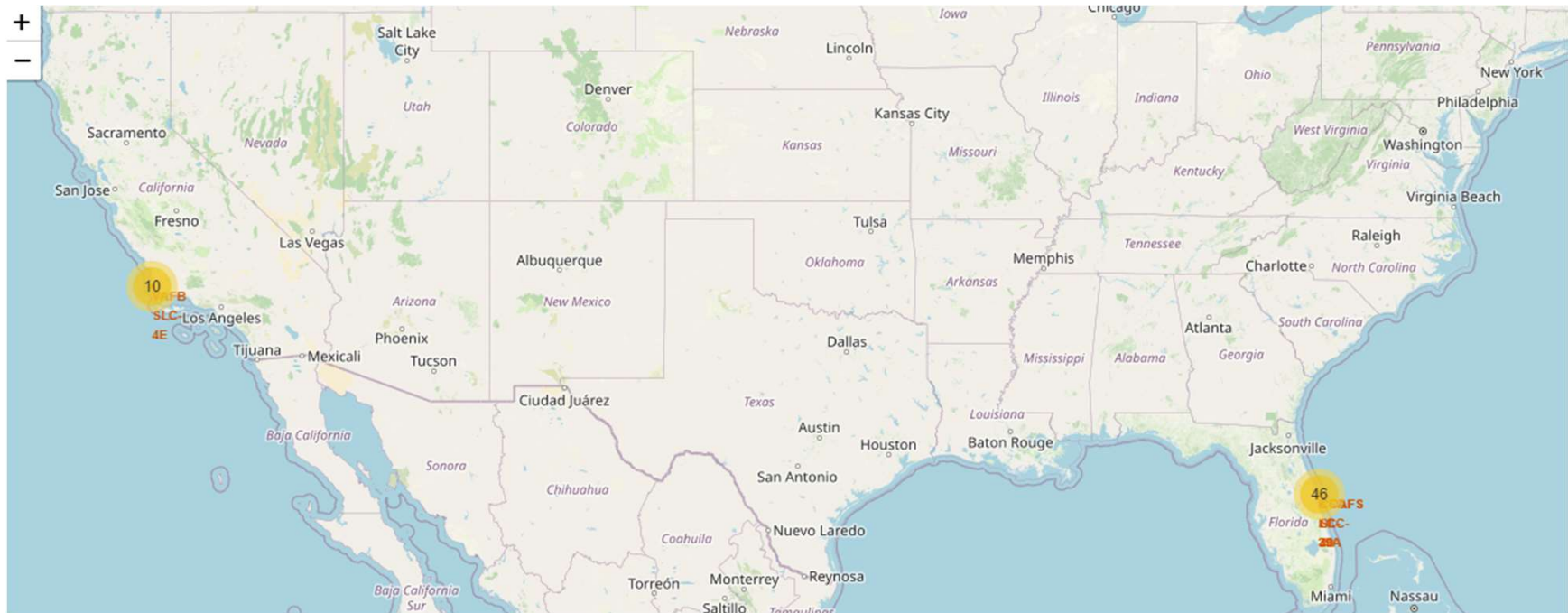
Task 1 Results

- Task 1 Mark all launch sites on a map



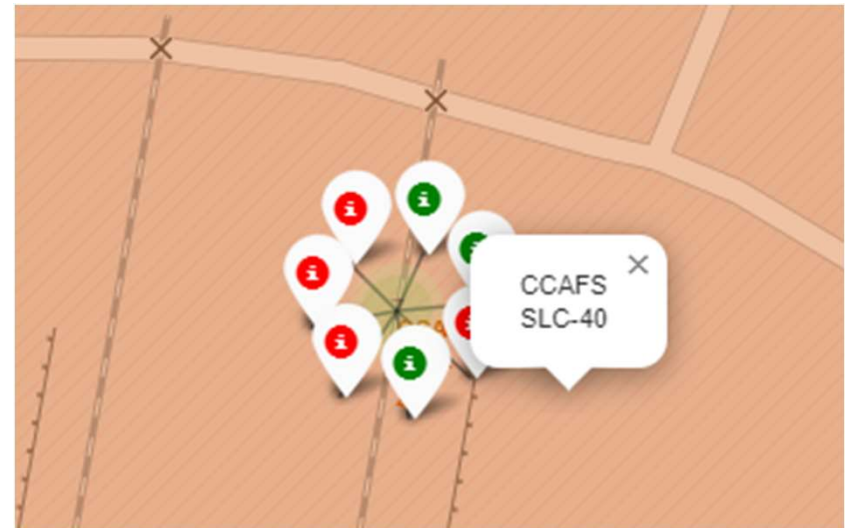
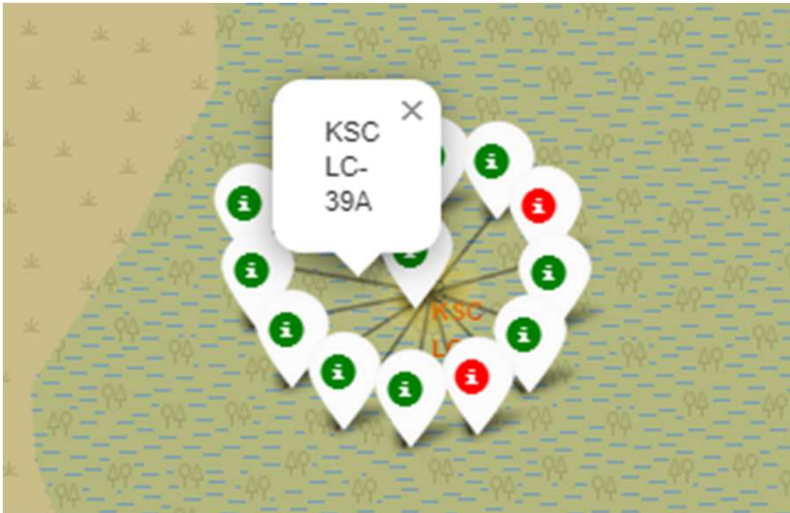
Task 1 Continued Results

- Task 1 Mark all launch sites on a map



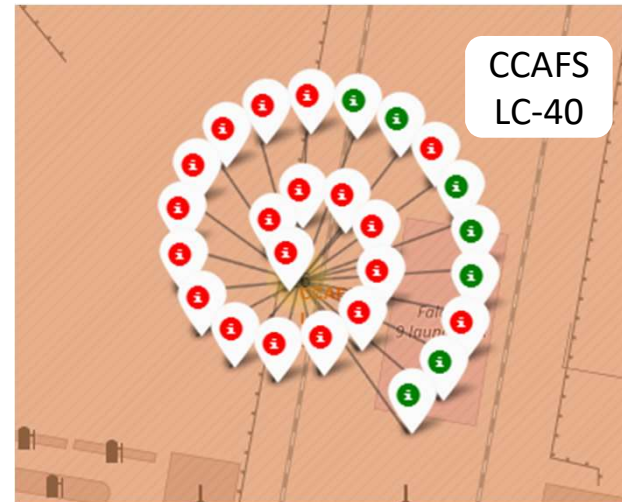
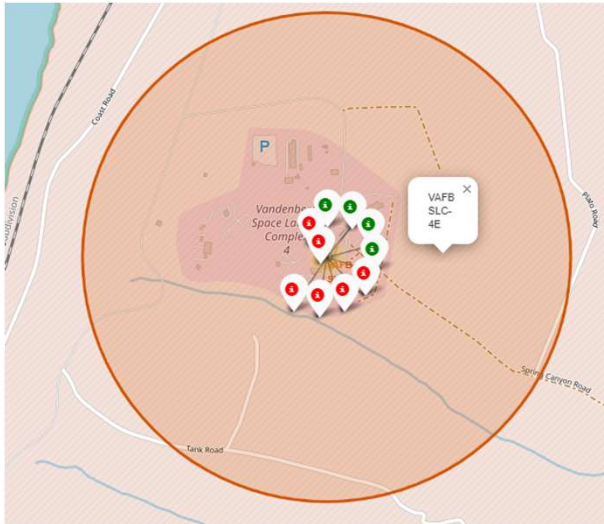
Task 2 Results

- Task 2 Mark all launch sites on a map



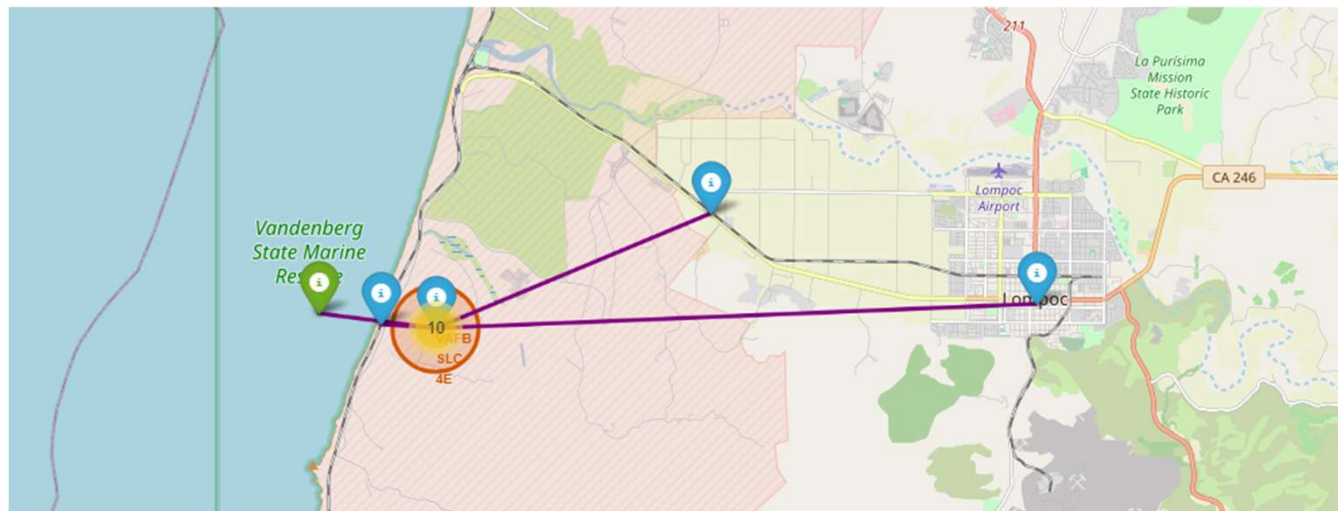
Task 2 Continued Results

- Task 2 Mark all launch sites on a map



Task 3 Results

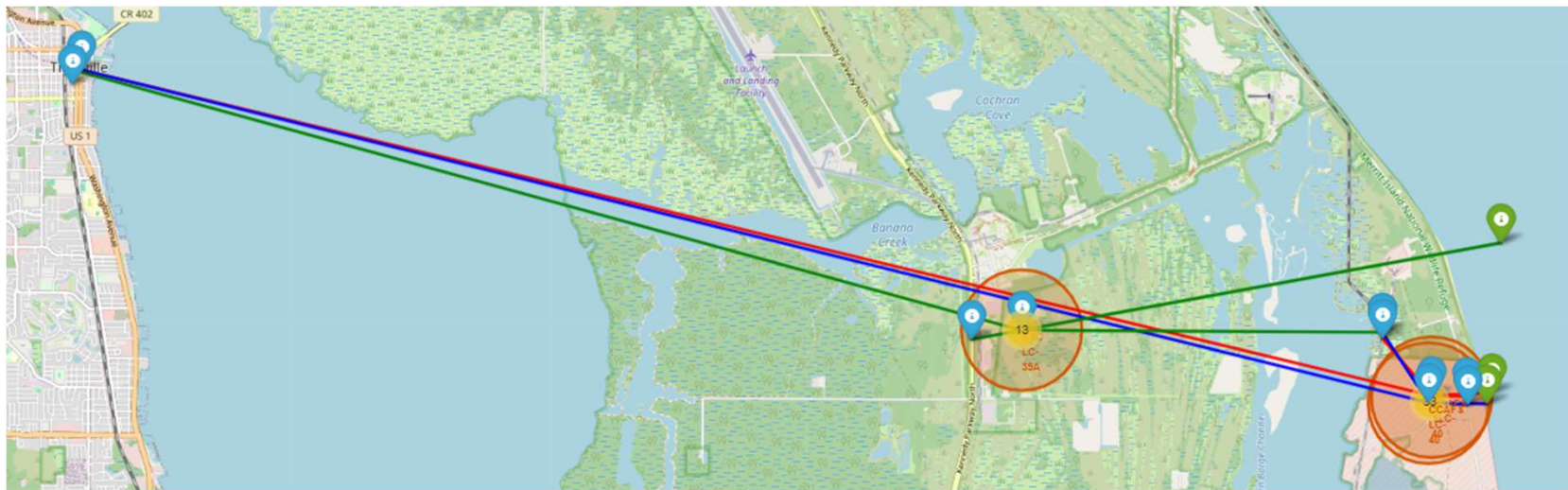
- Task 3 Calculate the distances between a launch site to its proximities



- Note:
 - The blue markers are the closest railroad, highway, and city to the launch site
 - The green marker the coastline point to the launch site
 - The purple line represents the distance between VAFB SLC-4E to its proximities

Task 3 Continued Results

- Task 3 Calculate the distances between a launch site to its proximities



- Note:
 - The blue markers are the closest railroad, highway, and city to the launch site
 - The green marker the coastline point to the launch site
 - The green line represents the distance between KSC LC-39 A to its proximities
 - The red line represents the distance between to CCFAS SLC-40 its proximities
 - The blue line represents the distance between CCFAS LC-40 to its proximities

Proximity Launch Site Questions Results

1. Are launch sites in close proximity to railways?

Ans: Yes, the launch sites try to stay within 1.5km from the railroad

2. Are launch sites in close proximity to highways?

Ans: Yes, the launch sites try to stay within 6km from highway

3. Are launch sites in close proximity to coastline?

Ans: Yes, the launch sites try to get as close to the coastline as possible. The goal is to be at least 2km away from the coastline, however, being further away from the coastline such as KSC LC-39A is manageable.

4. Do launch sites keep certain distance away from cities?

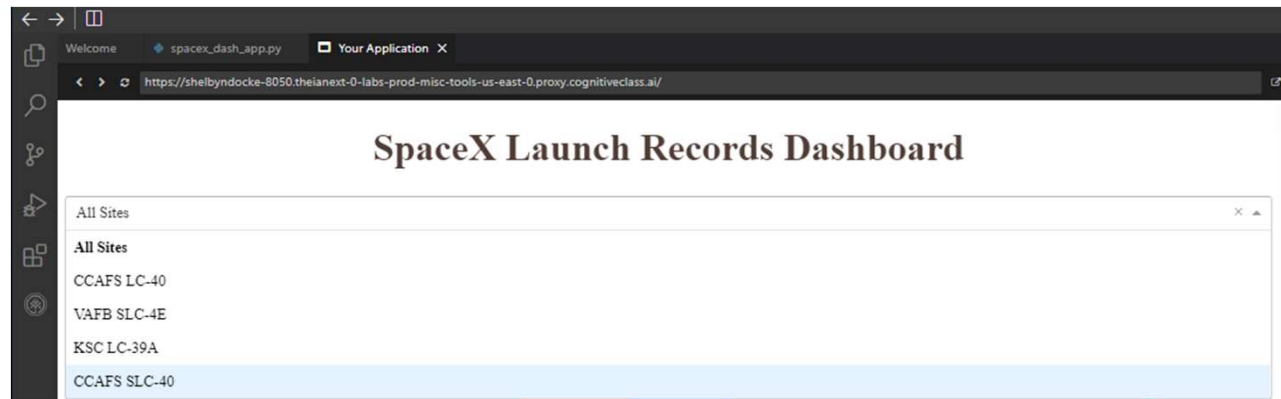
Ans: Yes, the launch sites try to keep at least 15km away from cities.

Plotly Dash dashboard results

- The following slides are plotly dash results from:
 - Plotly Dash Lab Tasks 1 through 4
 - Finding Insights Visually Questions:
 1. Which site has the largest successful launches?
 2. Which site has the highest launch success rate?
 3. Which payload range(s) has the highest launch success rate?
 4. Which payload range(s) has the lowest launch success rate?
 5. Which F9 Booster version (v1.0, v1.1, FT, B4, B5, etc.) has the highest?

Task 1 Results

Task 1: Add a Launch Site Drop-down Input Component



Task 2 Results

Task 2: Add a callback function to render success-pie-chart based on selected site dropdown

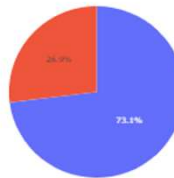
Total Success Launches by All Sites



0 1

KSC LC-39A
CCAFS LC-40
VAFB SLC-42
CCAFS SLC-42

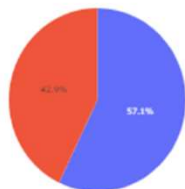
Total Success Launches for site CCAFS LC-40



0 1

0 1

Total Success Launches for site CCAFS SLC-40



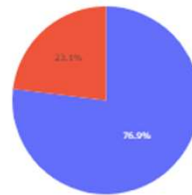
0 1

0 1

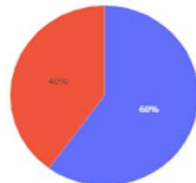
Task 2 Results Continued

Task 2: Add a callback function to render success-pie-chart based on selected site dropdown

Total Success Launches for site KSC LC-39A



Total Success Launches for site VAFB SLC-4E



Task 3 Results

Task 3: Add a Range Slider to Select Payload



Task 4 Results

Task 4: Add a callback function to render the success-payload-scatter-chart scatter plot



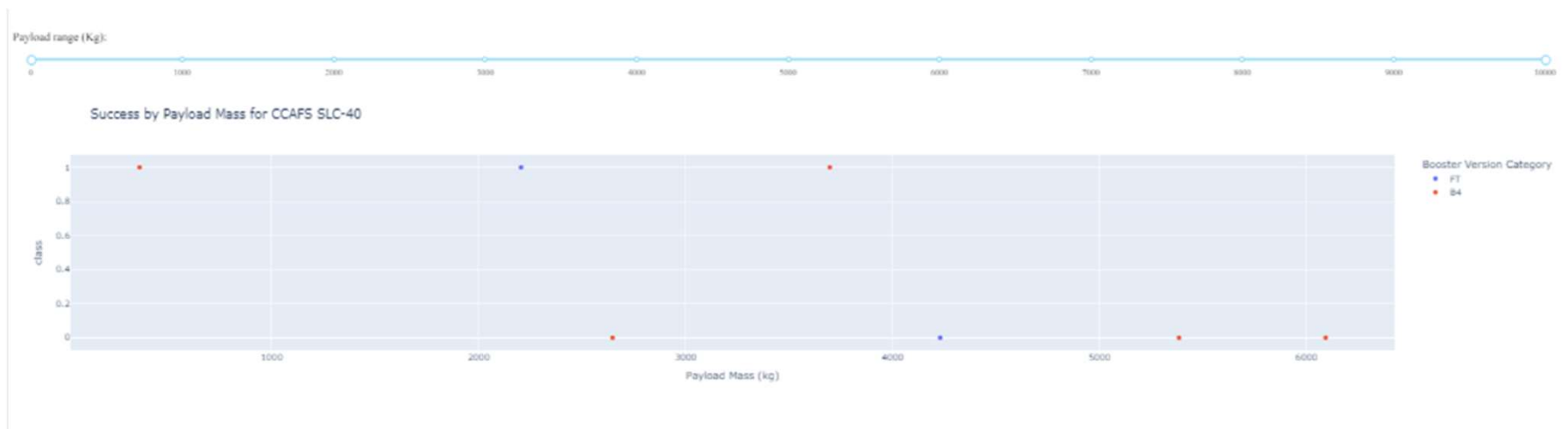
Task 4 Results Continued

Task 4: Add a callback function to render the success-payload-scatter-chart scatter plot



Task 4 Results Continued

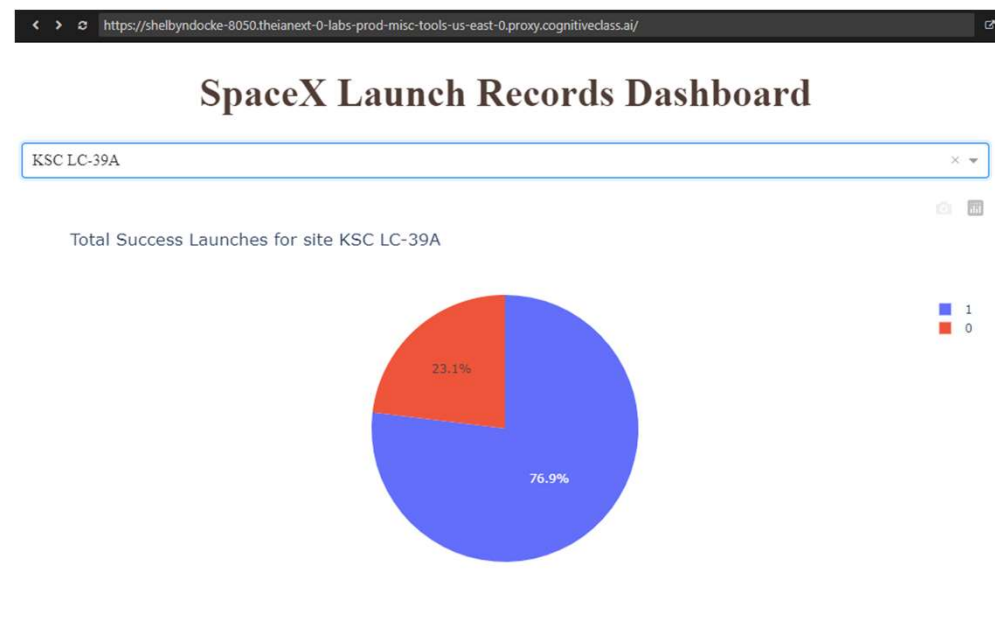
Task 4: Add a callback function to render the success-payload-scatter-chart scatter plot



Finding Insights Visually Question 1

- Which site has the largest successful launches?

KSC LC-39A

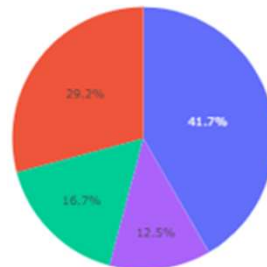


- Note:
 - Blue = Successful Launch
 - Red = Failed Launch

Finding Insights Visually Question 2

- Which site has the highest launch success rate?
KSC LC-39A
- The launch success rate for KSC LC-39A is 41.7%

Total Success Launches by All Sites

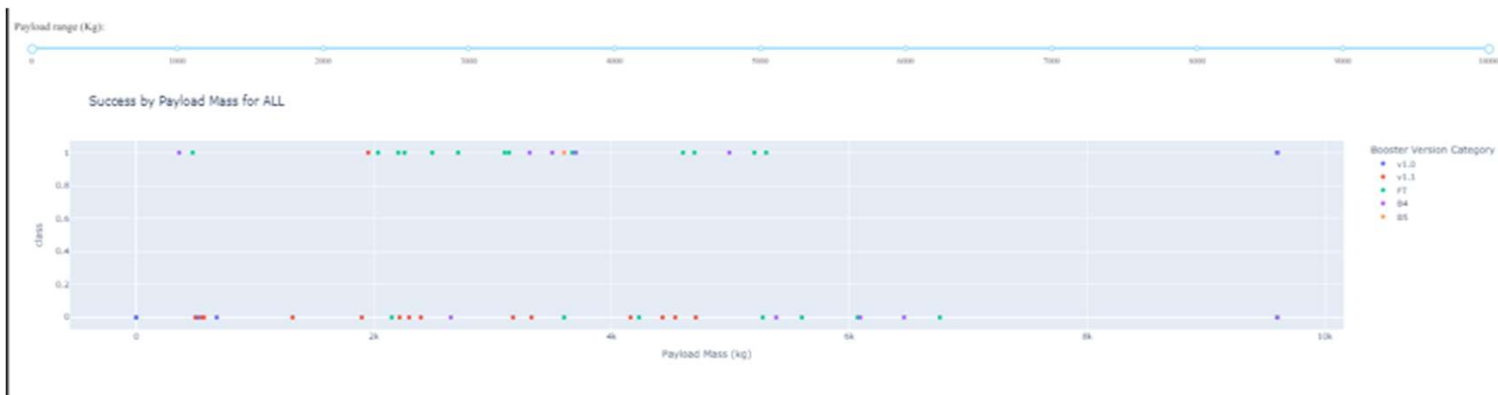


■ KSC LC-39A
■ CCAFS LC-10
■ VAFB SLC-1E
■ CCAFS SLC-10

Finding Insights Visually Question 3

- Which payload range(s) has the highest launch success rate?

Payload range between 2kg to 4kg



- Note:
 - The y-axis is class ranging from 0 to 1
1 = successful launch and 0 = failed launch
 - The x-axis is payload mass ranging from 0 kg to 10 kg
kg = kilograms

Finding Insights Visually Question 4

- Which payload range(s) has the lowest launch success rate?

Payload range between 2kg to 4kg

Payload range between 4kg to 6kg



- Note:
 - The y-axis is class ranging from 0 to 1
1 = successful launch and 0 = failed launch
 - The x-axis is payload mass ranging from 0 kg to 10 kg
kg = kilograms

Finding Insights Visually Question 5

- Which F9 Booster version (v1.0, v1.1, FT, B4, B5, etc.) has the highest launch success rate?

F9 Booster version FT



- Note:
 - The y-axis is class ranging from 0 to 1
1 = successful launch and 0 = failed launch
 - The x-axis is payload mass ranging from 0 kg to 10 kg
kg = kilograms

Predictive analysis (classification) results

- The following slides are Predictive analysis results from:
 - Predictive analysis Lab Tasks 1 through 12

Predictive analysis (classification) Task 1 results

- Task 1: Create a NumPy array from the column Class in data, by applying the method `to_numpy()` then assign it to the variable Y, make sure the output is a Pandas series (only one bracket `df['name of column']`).

```
[69]: print('task 1')
      print('getting Class col from data ')
      print('Class is now  in a panda series called Y')
      print()
      Y=data['Class'].to_numpy()
      Y=pd.Series(Y)
      print('end of task 1')
      print()
```

```
task 1
getting Class col from data
Class is now  in a panda series called Y

end of task 1
```

Predictive analysis (classification) Task 2 results

- Task 2: Standardize the data in X then reassign it to the variable X using the transform provided below.

```
[70]: # students get this
      #transform = preprocessing.StandardScaler()
      from sklearn.preprocessing import StandardScaler
      print('task 2')
      X = preprocessing.StandardScaler().fit(X).transform(X)
      print('Standardize the data in X')
      print('end of task 2')

task 2
Standardize the data in X
end of task 2
```

Predictive analysis (classification) Task 3 results

- Task 3: Use the function `train_test_split` to split the data X and Y into training and test data. Set the parameter `test_size` to 0.2 and `random_state` to 2. The training data and test data should be assigned to the following labels.

```
[71]: X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size=0.2, random_state=2)
      print('split test and training data')
      split test and training data
```


Predictive analysis (classification) Task 3 results

- Task 3: Use the function `train_test_split` to split the data `X` and `Y` into training and test data. Set the parameter `test_size` to 0.2 and `random_state` to 2. The training data and test data should be assigned to the following labels.

```
[71]: X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size=0.2, random_state=2)
      print('split test and training data')
      split test and training data
```

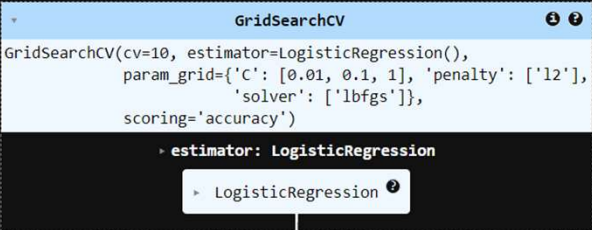
Predictive analysis (classification) Task 3 results

- Task 3: Use the function `train_test_split` to split the data `X` and `Y` into training and test data. Set the parameter `test_size` to 0.2 and `random_state` to 2. The training data and test data should be assigned to the following labels.

```
[71]: X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size=0.2, random_state=2)
      print('split test and training data')
      split test and training data
```

Predictive analysis (classification) Task 4 results

- Task 4: Create a logistic regression object then create a GridSearchCV object logreg_cv with cv = 10. Fit the object to find the best parameters from the dictionary parameters.

```
[90]: parameters = {'C': [0.01, 0.1, 1],  
                  'penalty': ['l2'],  
                  'solver': ['lbfgs']}  
  
[91]: #parameters = {'C': [0.01, 0.1, 1], 'penalty': ['l2'], 'solver': ['ridge']}# l1 lasso l2 ridge  
lr=LogisticRegression()  
  
[92]: logreg_cv = GridSearchCV(estimator=lr, param_grid=parameters, scoring='accuracy', cv=10)  
  
[93]: logreg_cv.fit(X_train, Y_train)  
  
[93]: 
```

```
[94]: print("tuned hpyerparameters :(best parameters) ", logreg_cv.best_params_)  
      print("accuracy :", logreg_cv.best_score_)  
  
tuned hpyerparameters :(best parameters) {'C': 0.01, 'penalty': 'l2', 'solver': 'lbfgs'}  
accuracy : 0.8464285714285713
```

Predictive analysis (classification) Task 5 results

- Task 5: Calculate the accuracy on the test data using the method score

```
[95]: # Evaluate other models (e.g., Logistic Regression, KNN)
      from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
      lr_accuracy = logreg_cv.score(X_test, Y_test)
      lr_precision = precision_score(Y_test, logreg_cv.predict(X_test))
      lr_recall = recall_score(Y_test, logreg_cv.predict(X_test))
      lr_f1 = f1_score(Y_test, logreg_cv.predict(X_test))
      print("Logistic Regression - Accuracy:", lr_accuracy, "Precision:", lr_precision, "Recall:", lr_recall, "F1 Score:", lr_f1)

Logistic Regression - Accuracy: 0.8333333333333334 Precision: 0.8 Recall: 1.0 F1 Score: 0.8888888888888888
```

Predictive analysis (classification) Task 6 results

- Task 6: Create a support vector machine object then create a GridSearchCV object `svm_cv` with `cv = 10`. Fit the object to find the best parameters from the dictionary parameters.

```
[108]: parameters = {'kernel': ('linear', 'rbf', 'poly', 'rbf', 'sigmoid'),
                    'C': np.logspace(-3, 3, 5),
                    'gamma': np.logspace(-3, 3, 5)}
svm = SVC()

[109]: #svm_cv = GridSearchCV(estimator=svm, param_grid=parameters, scoring='accuracy', cv=10)
svm_cv = GridSearchCV(estimator=svm, param_grid=parameters, cv=10)

[110]: svm_cv.fit(X_train, Y_train)

[110]: GridSearchCV
GridSearchCV(cv=10, estimator=SVC(),
             param_grid={'C': array([1.00000000e-03, 3.16227766e-02, 1.00000000e+00, 3.16227766e+01,
1.00000000e+03]),
             'gamma': array([1.00000000e-03, 3.16227766e-02, 1.00000000e+00, 3.16227766e+01,
1.00000000e+03]),
             'kernel': ('linear', 'rbf', 'poly', 'rbf', 'sigmoid')})
  estimator: SVC
    SVC()
      SVC
```

```
[111]: print("tuned hyperparameters : (best parameters) ", svm_cv.best_params_)
print("accuracy : ", svm_cv.best_score_)

tuned hyperparameters : (best parameters) {'C': 1.0, 'gamma': 0.03162277660168379, 'kernel': 'sigmoid'}
accuracy : 0.8482142857142856
```

Predictive analysis (classification) Task 7 results

- Task 7: Calculate the accuracy on the test data using the method score

```
[112]: from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
svm_accuracy = svm_cv.score(X_test, Y_test)
svm_precision = precision_score(Y_test, svm_cv.predict(X_test))
svm_recall = recall_score(Y_test, svm_cv.predict(X_test))
svm_f1 = f1_score(Y_test, svm_cv.predict(X_test))
print("support vector machine - Accuracy:", svm_accuracy, "Precision:", svm_precision, "Recall:", svm_recall, "F1 Score:", svm_f1)

support vector machine - Accuracy: 0.8333333333333334 Precision: 0.8 Recall: 1.0 F1 Score: 0.8888888888888888
```

Predictive analysis (classification) Task 8 results

- Task 8: Create a decision tree classifier object then create a GridSearchCV object tree_cv with cv = 10. Fit the object to find the best parameters from the dictionary parameters.

```
[51]: parameters = {'criterion': ['gini', 'entropy'],
                  'splitter': ['best', 'random'],
                  'max_depth': [2*n for n in range(1,10)],
                  #'max_features': ['auto', 'sqrt'],
                  'max_features': ['sqrt', 'log2'],
                  #'max_features': ['sqrt', 'log2', None],
                  'min_samples_leaf': [1, 2, 4],
                  'min_samples_split': [2, 5, 10]}

tree = DecisionTreeClassifier()

[52]: tree_cv = GridSearchCV(estimator=tree, param_grid=parameters, cv=10, error_score='raise')
tree_cv.fit(X_train, Y_train)

[52]: * GridSearchCV ⓘ ⓘ
GridSearchCV(cv=10, error_score='raise', estimator=DecisionTreeClassifier(),
            param_grid={'criterion': ['gini', 'entropy'],
                        'max_depth': [2, 4, 6, 8, 10, 12, 14, 16, 18],
                        'max_features': ['sqrt', 'log2'],
                        'min_samples_leaf': [1, 2, 4],
                        'min_samples_split': [2, 5, 10],
                        'splitter': ['best', 'random']})

* estimator: DecisionTreeClassifier
DecisionTreeClassifier()
* DecisionTreeClassifier ⓘ
DecisionTreeClassifier()

[53]: print("tuned hyperparameters :(best parameters) ",tree_cv.best_params_)
print("accuracy :",tree_cv.best_score_)

tuned hyperparameters :(best parameters) {'criterion': 'gini', 'max_depth': 4, 'max_features': 'sqrt', 'min_samples_leaf': 4, 'min_samples_split': 5, 'splitter': 'random'}
accuracy : 0.8767857142857143
```

Predictive analysis (classification) Task 9 results

- Task 9: Calculate the accuracy on the test data using the method score

```
[54]: dt_accuracy = tree_cv.score(X_test, Y_test)
      dt_precision = precision_score(Y_test, tree_cv.predict(X_test))
      dt_recall = recall_score(Y_test, tree_cv.predict(X_test))
      dt_f1 = f1_score(Y_test, tree_cv.predict(X_test))
      print("Decision Tree - Accuracy:", dt_accuracy, "Precision:", dt_precision, "Recall:", dt_recall, "F1 Score:", dt_f1)
```

```
Decision Tree - Accuracy: 0.8333333333333334 Precision: 0.8 Recall: 1.0 F1 Score: 0.8888888888888888
```


Predictive analysis (classification) Task 10 results

- Task 10: Create a k nearest neighbors object then create a GridSearchCV object knn_cv with cv = 10. Fit the object to find the best parameters from the dictionary parameters.

```
[56]: parameters = {'n_neighbors': [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
                  'algorithm': ['auto', 'ball_tree', 'kd_tree', 'brute'],
                  'p': [1, 2]}

KNN = KNeighborsClassifier()

[58]: knn_cv = GridSearchCV(KNN, parameters, cv=10)
      knn_cv.fit(X_train, Y_train)

[58]: 

GridSearchCV
      GridSearchCV(cv=10, estimator=KNeighborsClassifier(),
                  param_grid={'algorithm': ['auto', 'ball_tree', 'kd_tree', 'brute'],
                              'n_neighbors': [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
                              'p': [1, 2]})
      estimator: KNeighborsClassifier
      KNeighborsClassifier()
      KNeighborsClassifier



[59]: print("tuned hyperparameters : (best parameters) ", knn_cv.best_params_)
      print("accuracy : ", knn_cv.best_score_)

tuned hyperparameters : (best parameters) {'algorithm': 'auto', 'n_neighbors': 10, 'p': 1}
accuracy : 0.8482142857142858
```

Predictive analysis (classification) Task 11 results

- Task 11: Calculate the accuracy on the test data using the method score

```
[114]: knn_accuracy = knn_cv.score(X_test, Y_test)
knn_precision = precision_score(Y_test, knn_cv.predict(X_test))
knn_recall = recall_score(Y_test, knn_cv.predict(X_test))
knn_f1 = f1_score(Y_test, knn_cv.predict(X_test))
print("k nearest neighbors - Accuracy:", knn_accuracy, "Precision:", knn_precision, "Recall:", knn_recall, "F1 Score:", knn_f1)

k nearest neighbors - Accuracy: 0.8333333333333334 Precision: 0.8 Recall: 1.0 F1 Score: 0.8888888888888888
```

Predictive analysis (classification) Task 12 results

- Task 12: Find the method that performs best
- To best honest, I believe the decision tree is performs the best because its best_score is the highest compare to the rest, however, I am not 100% certain on that since for some reason using the score method to calculate the accuracy for each method I get an accuracy of 83%. Therefore, according to the data it does not matter which model you use. However, if I were to choose a model, it would be the Logistic Regression.

Conclusion

- This report has presented the findings of a data analysis project about predicting the success of the first stage landing of the SpaceX Falcon 9 rocket. The reason behind this data analysis project was to assist a startup company in making informed bids against SpaceX for rocket launches.
- The project involved several data analysis and data science skills such as data collecting and data wrangling, performing exploratory data analysis, developing predictive models, and evaluating. The dataset contain the historical data on the Falcon 9 launches.
- The results of this analysis can provide some insights for a startup company to make data-driven decisions when bidding against SpaceX. By using the predictive model, the company can assess the likelihood of a successful landing and adjust their bids accordingly.